2025 IEEE International Symposium on High Performance Computer Architecture (HPCA) | 979-8-3315-0647-6/25/$31.00 ©2025 IEEE | DOI: 10.1109/HPCA61900.2025.00128

Lincoln: Real-Time 50∼100B LLM Inference on Consumer Devices with LPDDR-Interfaced, Compute-Enabled Flash Memory

Weiyi Sun[1], Mingyu Gao[2], Zhaoshi Li[3], Aoyang Zhang[4], Iris Ying Chou[1],
Jianfeng Zhu*[4], Shaojun Wei[4], Leibo Liu*[4]
School of Integrated Circuits, Tsinghua University, Beijing, China
Beijing National Research Center for Information Science and Technology (BNRist)[1,4],
Institute for Interdisciplinary Information Sciences, Tsinghua University, China
Shanghai Qi Zhi Institute, Shanghai, China[2],
MetaX Technology, Beijing, China[3]
Email: {sunwy19, yzhou22}@mails.tsinghua.edu.cn[1],
{gaomy, aoyang, jfzhu, wsj, liulb}@tsinghua.edu.cn[2,4], zhaoshi.li@metax-tech.com[3]

***Abstract*—With the widespread use of large language models (LLMs), and with the privacy and cost concerns on cloud-based services, vendors are now pushing LLM inference to consumer devices. However, current attempts only enable real-time inference of low-quality small-sized LLMs. Large-sized LLMs have to load most of their weights from Flash storage for every execution iteration, which dominates the execution time of both the prefill and the generation phase. This performance bottleneck is attributed to both the low internal Flash memory bandwidth and the low transmission bandwidth between Flash and the Neural Processing Unit (NPU). To tackle these two challenges, we present Lincoln, a device-architecture co-design solution with LPDDR-interfaced, Compute-Enabled Flash Memory.**

On the device level, we boost the Flash internal bandwidth by improving upon existing array shrinking methods, to enable lower read latency and more parallel Flash planes within each Flash die. We specifically leverage 3D hybrid bonding, which is already adopted in consumer Flash products, to maintain high area efficiency and low density loss. On the architecture level, to leverage such increased internal bandwidth for resolving the transmission bottleneck, we propose two solutions for the two distinct phases of LLMs. For the compute-intensive prefill phase, we let Flash devices use the existing high-speed LPDDR interface (originally for DRAM), which offers much higher transmission bandwidth to the NPU than the conventional Flash interface, while maintaining good cost and area efficiency. For the memory-intensive generation phase, we rely on hybrid-bonding-based near-Flash computing to fully utilize the internal Flash bandwidth, and further equip with speculative decoding to eventually reach the real-time latency goal.

Our evaluation shows that Lincoln enables real-time inference, with up to $13.23\times$ and $254.1\times$ speedups for LLM prefill and generation phases over conventional SSD-based systems.

I. Introduction

With recent revolutionary improvements, large language models (LLMs) are becoming increasingly indispensable in people's everyday life [94]. However, current LLM services are mainly maintained in clouds due to their high demands on the compute platforms [75] [31] [83], which brings several major concerns. On the user side, there are doubts on clouds' privacy protection. The long queuing delays and service failures in congestion hours also seriously harm user experience. On the vendor side, high inference infrastructure ownership and maintenance costs are also frustrating major LLM service providers. For small startups, such costs easily exceed their budgets.

With these in mind, many vendors (e.g. Apple [6], OPPO [59], Intel [35], Qualcomm [82], etc.) are pushing LLM inference to consumer devices, like personal computers, laptops, and even mobile phones. Yet, even with the help of aggressive quantization, only small-sized (<13B) LLMs are supported [59] [82] for real-time inference scenarios like chatbots, while larger-sized LLMs are only possible for time-insensitive tasks. Such practices cannot provide satisfactory user experience, since the size scaling law plays a major role in LLM service quality (e.g. Llama-3.1 8B/70B's scores on MMLU-Pro, MATH, GPQA, Nexus and MGSM are 48.3/66.4, 51.9/68.0, 32.8/46.7, 38.5/56.7, 68.9/86.9) [22], and techniques like quantization and pruning only deteriorate the quality issues. Therefore, in this paper, we strive to enable real-time 50∼100B LLM inference on consumer devices while incurring no harm to the original accuracy.

The culprit of poor consumer-side 50∼100B LLM inference ability lies in the low performance of Flash storage. Due to the form factor limitations and the trend for compactness, consumer devices like laptops mostly possess ≤64GB DRAM. Thus, for large-sized LLM weights, most of them can only reside in the Flash storage, and have to be read from Flash to DRAM for every iteration of model inference (i.e. capacity thrashing). With over $10\times$ lower bandwidth than DRAM's, such data loading completely dominates the model execution time, not only for the memory-intensive LLM generation phase, but even for the traditionally compute-intensive prefill phase.

Two major factors contribute to Flash's low performance. First, the transmission interface between the Neural Processing Unit (NPU) and Flash bears low bandwidth. In fact, most of the high-speed off-chip bandwidth resources of the processor

*Corresponding authors: Leibo Liu (liulb@tsinghua.edu.cn) and Jianfeng Zhu (jfzhu@tsinghua.edu.cn)

979-8-3315-0647-6/25/$31.00 ©2025 IEEE
DOI 10.1109/HPCA61900.2025.00128

chip are dedicated to DRAM in conventional consumer device designs, while the Flash package is allocated with much fewer transmission pins (e.g. 128 pins and more for LPDDR data transfer, while only around 4-8 lanes for NAND Flash [92] [78]). This is reasonable, since Flash is conventionally inferior to DRAM in memory hierarchy. Second, the internal bandwidth of Flash memory arrays is low [17]. Traditionally, Flash products are driven by density. This leads to large array designs that result in high latency and low parallelism, and thus low performance. Even with unlimited transmission bandwidth, such low internal bandwidth would still bottleneck the overall execution time.

To conquer both limitations and meet the stringent real-time requirement, we must optimize at both the device and the architecture level, with the device-level design alleviating the internal bandwidth limitation, and the architecture-level design mitigating the transmission bandwidth limitation. Thus, we present Lincoln, a device-architecture co-design solution with LPDDR-interfaced, compute-enabled Flash memory.

On the device level, we follow and improve the existing array shrinking method, which implements smaller Single-Level-Cell (SLC) arrays that result in smaller read latency ($<$4us) and more arrays (up to 32 planes) in one Flash die [50] [44]. By making more planes work in parallel with shorter latency, we achieve $\sim$34GBps read bandwidth per die and $\sim$500GBps with 16 dies. Specifically, we base our design on the commercial array-shrunk SLC products like XL-Flash [50] [49], and further utilize the hybrid bonding technology to hide peripheral overheads, i.e., 3D-stacking a logic layer of peripheral circuitry beneath the Flash layer. This enables more aggressive array shrinking and thus $2\times$ plane count without increasing die size or harming storage density. The related cost is acceptable, considering that hybrid bonding is already commercially adopted for consumer Flash products [69] [101] [72]. Besides, we also consider the power supply issue concerning parallel plane operations in our design.

On the architecture level, to resolve the transmission bottleneck of LLM inference and better utilize the boosted Flash internal bandwidth above, we propose to **(1)** leverage the existing high-speed LPDDR DRAM interfaces/packages to enable high-bandwidth data transfer from Flash to compute-rich NPU for the compute-intensive prefill phase (similar to a mobile version of NVDIMM-F [3] [104]); and **(2)** implement ECC-enabled near-Flash logic through hybrid-bonding-based 3D stacking to enable full utilization of Flash internal bandwidth for the memory-intensive generation phase.

(1) Specifically, the LPDDR package/interface enables much higher bandwidth ($>$100 GBps) than the conventional Flash interface ($\sim$10 GBps), while maintaining good area efficiency through low-cost die stacking. We thus abandon the conventional Flash package/interface (e.g. NVMe/UFS), and vertically place Flash dies inside the LPDDR-style package along with DRAM dies. Acting as extra DRAM ranks, these Flash dies share existing LPDDR channels with the DRAM dies, thus enabling fast Flash-NPU data transfer during LLM prefill phase, while minimizing implementation cost through reuse of existing channels.

Yet, to realize LPDDR-based interface, the high NAND Flash access cost is a major obstacle, including both long Flash-side read latency (us) and large controller-side ECC runtime overheads for reads. Specifically, the former can cause violation of LPDDR timing ($<$100ns), while both can greatly degrade performance. For resolution, we adopt the following methods. First, we introduce a low-latency Flash-side SRAM buffer, which stages data fetched from Flash arrays and then serves LPDDR accesses without timing violation. Second, we implement double buffering for the introduced buffer, and leverage the high predictability of LLM's memory access patterns, to overlap Flash-side latency with computation. Third, we utilize the near-Flash ECC modules, already introduced for near-Flash computing, to replace controller-side ECC. This could both avoid ECC-related LPDDR access costs and also incorporate the remaining overheads within the Flash read process, which can be hidden through the double buffering above.

(2) For the compute-light but memory-intensive LLM generation phase, however, the LPDDR interface above is still not enough to deliver acceptable performance. Fortunately, the Flash internal bandwidth ($>$500GBps) has not been fully utilized. Besides, the logic layer for peripheral, as has been introduced by hybrid bonding, is also generally not fully utilized [20], and provides opportunities for implementing customized light-weight logic [20] [19]. Thus, we can implement near-Flash compute units and necessary ECC engines [102] to fully leverage the Flash internal bandwidth and boost the generation phase performance without much overhead. Such logic is generally not suitable for the compute-intensive prefill phase, though, since it is too light-weight to meet the compute requirement.

Yet, to utilize the near-Flash logic, four issues need to be considered. First, real-time generation latency may not be achieved even with fully utilized Flash internal bandwidth. Specifically, for an 8-channel 16-die Lincoln system ($\sim$500GBps), one execution iteration of a $>$100B model ($>$200GB) easily reaches 0.4s, much beyond the generation latency constraint [107]. For resolution, we further enable the system to utilize speculative decoding [58] [63], an algorithmic technique that enables multi-token generation per iteration without accuracy loss, so as to eventually reach the real-time performance goal.

Second, the weight matrices stored in the Flash die need to serve both the near-Flash logic and NPU efficiently. Specifically, due to the relatively moderate capacity of SLC Flash ($<$300GB with 2 packages) compared to large-size LLMs (e.g. $>$75B model), keeping two separate copies of weights in different physical layouts may not be preferable. For this challenge, we leverage both a round-robin matrix row/column distribution scheme among Flash dies and the aforementioned Flash-side latency-hiding buffer. The former enables full utilization of memory channel parallelism by the NPU, while still preserving simple and efficient near-Flash computation. The latter enables the NPU to access inner tiles of a large enough weight partition at random with no performance loss, thus preserving enough flexibility for efficient NPU computation.

Third, the frequent and high-speed near-Flash computing can trigger serious read disturb issues. For this issue, we show that it can be resolved using read reclaim, i.e., using programs/erases (P/E) to reduce read disturb errors, similar to data refresh [12]

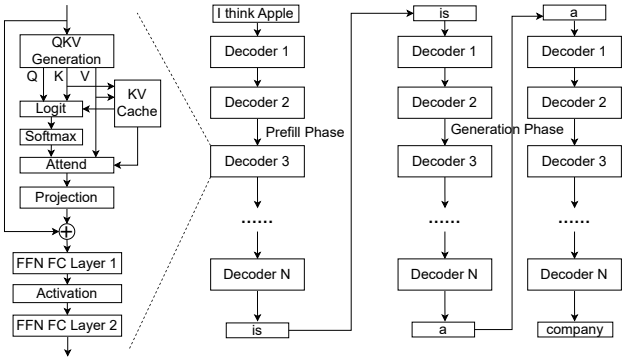

Fig. 1: LLM architecture and execution flow.

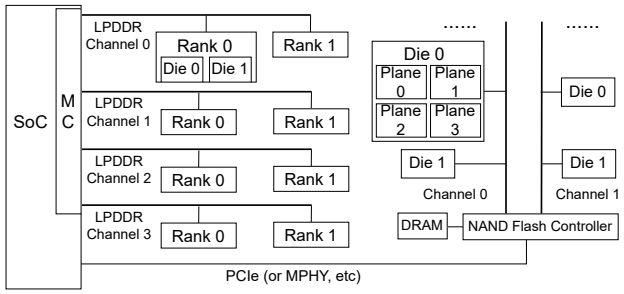

Fig. 2: Memory and storage systems on consumer devices.

[13]. The resulting number of P/E cycles stays well below the endurance threshold. Fourth, the vertical stacking may cause heat dissipation concerns. Our evaluation shows that heat dissipation is fully acceptable, causing only mild temperature increase.

In summary, we make the following contributions:

- We identify both the transmission interface and internal bandwidth of Flash as performance bottlenecks of LLM inference on consumer devices, for both the prefill and generation phases. Thus we present Lincoln, a device-architecture codesign to alleviate these bottlenecks.
- On the device level, we follow the array shrinking method for lower latency and more parallelism to boost the Flash internal bandwidth, while improving storage density with hybrid bonding and also considering power supply issues.
- On the architecture level, we propose to utilize LPDDR interface to mitigate the transmission bottleneck in the prefill phase, while utilizing near-Flash logic to fully leverage the Flash internal bandwidth in the generation phase, which is further enhanced with speculative decoding. Related issues like latency, data layout, endurance, heat dissipation, and control system design are also considered in our design.
- Our evaluation shows that Lincoln enables real-time 50∼100B LLM inference on consumer devices, with up to $13.23\times$ and $254.1\times$ speedups for the prefill and the generation phase over conventional SSD-based systems.

II. Background

A. Large Language Models

Model Architecture. Large language models (LLMs) generally consist of many layers of decoder blocks [96], as shown in Figure 1. The majority of model weights are used in the Fully-Connected (FC) layers, i.e. FCs for Query/Key/Value (Q/K/V) generation, Projecton, and the Feed-Forward Networks (FFNs), which essentially only involve matrix-matrix/vector multiplication (GEMM/GEMV). For attention computation, dot products of Query and Key vectors are first computed with causal masks. The results are then processed with softmax, with which the weighted sums of the Value vectors are calculated, and are finally fed to the projection FC.

Model Execution. LLM execution involves two phases — prefill and generation [96]. It begins with prefill, which works on the sequence of user input tokens and involves only one execution iteration of the model. The FCs are applied to all tokens, and thus are mainly GEMMs. The K/Vs generated in this phase are kept in a temporary data structure called KV Cache. Then in the generation phase, models are executed through many iterations, each generating an output token, which is then fed as the input of the next iteration. With only one token, the FCs here are GEMV. Attention is calculated through the entire KV Cache, and the newly generated KVs are added to the Cache. In scenarios like multi-round conversations, the prefill phase becomes a more general append-style prefill, which not only calculates attention among the input tokens, but also on the KV cache that stores KVs of previous conversation rounds.

B. Memory/Storage Systems on Consumer Devices

On consumer devices, the SoC (System-on-Chip, including CPU, GPU, NPU, etc.) connects to the DRAM system and the Flash storage system through off-chip links. Due to the trend for compactness, LPDDR is now widely used for the DRAM system. As shown in Figure 2, SoC's LPDDR Memory Controller (MC) connects to multiple channel buses, each with one or two ranks that consist of one or two LPDDR DRAM dies [77]. Each channel has 16 pins, and up to 8 or more channels can be integrated in consumer devices, leading to >100GB/s bandwidth. Physically, DRAM dies are stacked inside the LPDDR package(s). Each package can contain 4∼32 dies, and 2∼8 channels [77].

The Flash storage devices are often placed farther away from the SoC than DRAM. They are also connected to the controller through multiple channels, each with multiple dies [92] [66] [65]. On each die, there are a number of Flash planes (often 2 or 4 planes) that may or may not be able to operate in parallel, a bit similar to the bank concept in DRAM [17]. However, unlike DRAM, the controller does not stay on the SoC. Rather, it is connected to the SoC through serial links (e.g. PCIe for NVMe protocol or MPHY for UFS protocol), and the pins allocated to this link are generally much fewer than those of DRAM channels (4 lanes for PCIe or 2 lanes for MPHY), and thus with a much smaller external I/O bandwidth (∼10GB/s).

C. Hybrid Bonding for Flash

Wafer-to-wafer hybrid bonding [69] [101] [72] enables high-performance 3D-stacking by connecting the metal layers of two dies. Without the use of Through-Silicon Vias (TSVs), it can yield lower costs than TSV-based stacking. For 3D NAND Flash,

it maximizes the storage density by placing the peripheral circuits on a separate CMOS logic die, 3D-stacked with the Flash array die. This also enables peripheral circuitry of higher performance and NAND Flash cells of better quality, since peripherals can now be produced independently and thus completely with the performance-oriented normal CMOS process, eliminating disturbance caused by the NAND Flash manufacturing process, and the same also applies to the Flash arrays [69] [101]. It has been successfully commercialized by YTMC [1], which well shows that the costs stay within the stringent NAND Flash production budget. It is also expected to be more widely adopted in coming NAND Flash generations [69].

III. Motivation

A. Flash Storage Bottlenecks in LLM Execution

The key bottleneck of LLM execution on consumer devices lies in the Flash storage. Due to their stringent form factor limitation, consumer devices rarely have DRAM capacity of larger than 64GB, while >50B and >100B LLMs boast sizes of >100GB and >200GB, well beyond what DRAM can offer. Thus, for large-sized LLMs (>50B), only a small fraction of the weights can be loaded in DRAM on consumer devices at a time, while most of them have to stay in Flash. Furthermore, since each LLM execution iteration has to go through the entire weights, most LLM weights have to be repeatedly loaded from Flash for every prefill phase, and for every output token produced in the generation phase. Considering mainstream Flash devices' poor I/O bandwidth of merely several GB/s, we can easily foresee that these loading processes bottleneck the overall performance.

Figure 3 shows LLM execution time breakdown of an NPU equipped with a traditional NVMe-based Flash SSD and 2 LPDDR packages (Base-2), as detailed in Sec. V-A. For generation phase, the latency per output token is up to 41.8s, with 94.0% of the time contributed by the Flash-to-DRAM weight loading procedure on average. The same stands for the prefill phase, with 91.4% of its latency occupied by weight loading. Such long latency renders current consumer devices far from satisfying real-time inference requirements of large-sized LLMs.

Furthermore, unlike the cases with DRAM and PIM [75] [31] [83], resolving this I/O bottleneck is still away from reaching the real-time latency goal. Figure 3 also shows the execution time breakdown of an NPU with unlimited I/O for its Flash storage. Here, we assume zero latency and infinite bandwidth for all related I/O interfaces, including PCIe, Flash channels and even the bus I/Os within the Flash die. Thus the performance limiting factors only include the internal read latency and the total number of Flash planes. Still, the Flash storage contributes to a significant portion of the execution, even for the prefill phase, making them still far away from meeting the real-time requirement. Specifically, this is due to the internal read latency of the Flash arrays. For conventional density-oriented Flash array designs, a latency of over 50 us (and up to >100us) has to be taken for each 16KB page read. Thus even with 32 dies and 4 planes per die at hand, no more than 41GB/s internal bandwidth can be obtained, far from reaching LLM's bandwidth demand.

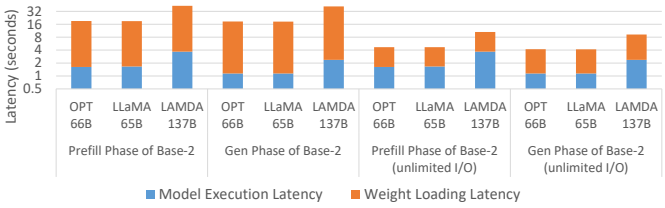

Fig. 3: LLM execution time breakdown for Base-2, and Base-2 with unlimited I/O (shown in logarithmic scale).

With all these in mind, we therefore conclude that, to reach our goal of real-time inference for 50∼100B LLM inference on consumer devices, we need to overcome both the internal Flash bandwidth limitation and the Flash I/O constraint.

B. Opportunities to Reach Real-Time Latency Goal

For space-limited consumer devices and capacity-demanding LLM weights, density is a critical issue. Since other low-latency storage technologies generally cannot provide as high a density as 3D NAND Flash, we stick to the use of Flash in our design, and observe the following opportunities to overcome the challenges above, from the device to the architecture level.

Device level: Shrinking Flash array size for shorter latency and more parallelism. An effective method for lowering Flash read latency is to implement smaller Single-Level-Cell (SLC) Flash arrays, as already adopted by commercial products like XL-Flash [50] [49]. Such a design lowers the parasitic capacitance and resistance of bitlines/wordlines, thus shortening the related precharge/discharge/activation time that dominates read latency [76] [65]. Furthermore, a smaller size also means that more independent Flash planes can be placed in one die, which can work in parallel [44]. By leveraging the reduced latency and the higher parallelism, a higher internal bandwidth can thus be achieved. Yet, the relative increase in peripheral circuit area would result in density downgradation. This will be resolved with hybrid-bonding-based 3D stacking in our design, as has been commercially adopted for consumer-grade NAND Flash products [1] [101]. Besides, the power supply issue related to parallel plane operations will also be considered.

Architecture level: Utilizing LPDDR interface for I/O bottleneck mitigation for the prefill phase. For the compute-intensive prefill phase, it is necessary to move weights out of Flash and onto the NPU. To speedup this movement, some naive options exist but are not applicable. First, increasing the transmission pins for NAND Flash will lead to large area/cost overhead on the NPU SoC for the increase in PHYs, possibly violating the area/cost restrictions on consumer devices. Second, utilizing 2.5D packaging causes unacceptable costs. Generally, up to 16 Flash dies are required to provide the desired storage capacity, each with >50 mm^2 area. To interface with these many large dies within acceptable area, 2.5D packaging has to rely on Through Silicon Via (TSV) based 3D stacking, as with HBM, since only TSV can provide sufficiently efficient connection between 2.5D links and stacked dies. This is prohibitively costly, considering that HBM has hardly reached the consumer market.

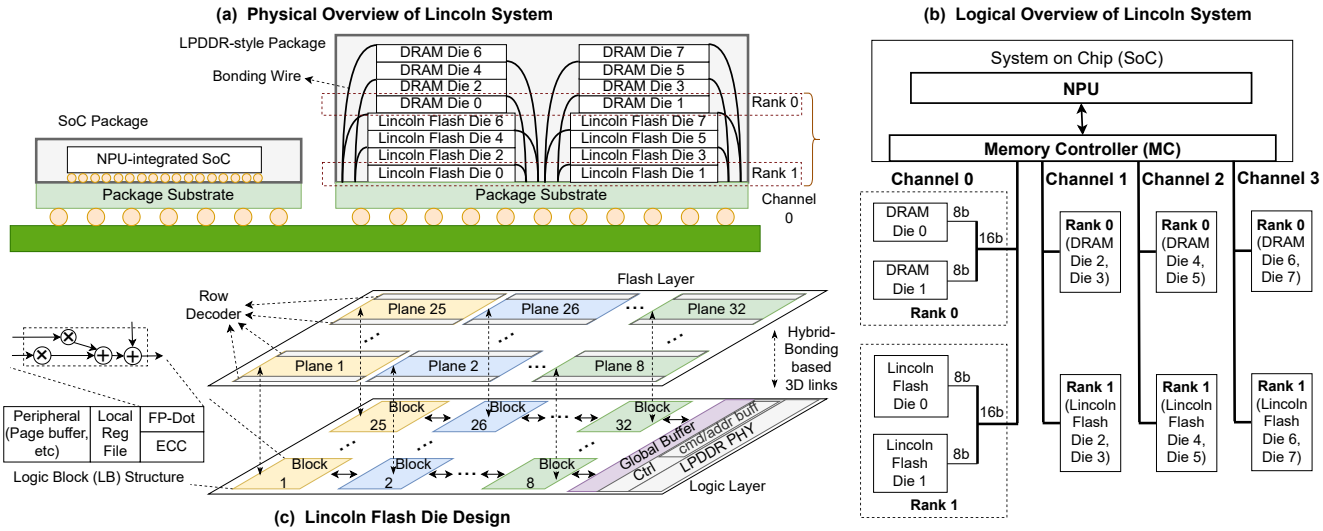

Fig. 4: (a) Physical and (b) logical overview of the Lincoln system. (c) Design schematic of the Lincoln Flash die.

In contrast, the LPDDR interface satisfies all the three requirements for performance, cost and form factor. First, LPDDR already exists in current consumer devices [78], and the LPDDR channels can be shared or reused by multiple dies, thus minimizing costs [77]. Second, since a majority of off-chip I/O bandwidth resources of consumer-device SoCs are allocated to LPDDR, it provides the highest performance among existing on-SoC interfaces [78]. Third, LPDDR-style packaging allows cheap vertical stacking of multiple dies with simple wire bonding [77] rather than costly TSV, thus mitigating the form factor overheads while keeping costs low.

Architecture level: Leveraging near-Flash computing to remove I/O bottleneck for the generation phase. For memory-intensive tasks like the LLM generation phase, even using LPDDR interface is still insufficient [83]. To tackle this problem, we make two observations. First, our array-shrunk Flash is actually able to provide even higher internal bandwidth than the LPDDR interface, which has not yet been fully utilized. Second, since we (as well as the industry) already utilize hybrid bonding to cope with the density issues, utilizing the logic layer for further compute capability adds only limited costs. Thus, by placing light-weight compute units required by the generation phase on the logic layer of the Flash package, it is possible to remove the need for bandwidth-constraint off-chip transmission, fully utilizing the internal array bandwidth for performance boost. Moreover, since speculative decoding [58] [100] allows generation of multiple tokens in one iteration without accuracy loss, the near-Flash logic can be equipped with such support to further enhance performance. Finally, it should be noted that near-Flash computing can only be used for the compute-light generation phase, since the compute capability of near-Flash logic is highly limited. For the compute-intensive prefill phase, the LPDDR-based NPU-side computation is indispensable.

With all the opportunities above in mind, we present Lincoln, a device-architecture co-design to reach real-time latency goal.

IV. Lincoln System Design

A. Overview

LPDDR-based Packaging and Interconnect Architecture. To increase transmission bandwidth, the architecture of Lincoln abandons the standard Flash package, and heavily relies on the existing LPDDR memory interface [77]. Figure 4(a) shows the physical overview of the Lincoln system design, which consists of the NPU-integrated SoC and LPDDR packages, connected through the PCB/on-package wires. Within the LPDDR package, four LPDDR-Interfaced, Compute-Enabled (Lincoln) Flash dies are vertically stacked, along with four DRAM dies. Two such vertical stacks are incorporated in one package, and all the dies are connected to the package pins through bonding wires. Correspondingly, Figure 4(b) shows the logical view of Lincoln. The memory controller (MC) of SoC connects to the 4 channels provided by the package shown in Figure 4(a), each with one Flash rank and one normal DRAM rank. A rank consists of two dies, each connected to 8 DQ pins of the 16-bit channel. For illustration, Figure 4(a) shows the 4 dies that respectively belong to the two ranks in Channel 0 with dashed boxes. Such a packaging configuration generally follows the LPDDR package design of [77], and we conservatively assume fewer channels per package than the latest design [77] to account for the differences of DRAM and Flash dies. Besides, note that the DRAM and the Flash ranks here compose a flat memory space where both are directly accessible by software; the DRAM is for common use, *rather than* used as a simple cache for the Flash.

The overall design above enables the Flash dies to connect to the LPDDR interface, while achieving our goals for minimized cost and form factor overhead. For one thing, we do not add extra channels for the Flash dies. Instead, we have them act as one extra rank within the existing LPDDR channels, and share the existing channel pins with the other DRAM dies. Compared to allocating entire channels to Flash, such sharing avoids the waste of channel bandwidth and thus maintains DRAM system performance when

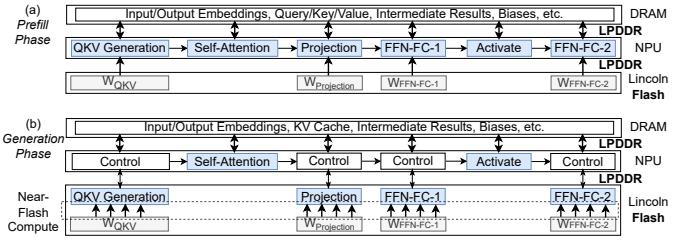

Fig. 5: Lincoln's overall dataflow

there is no LLM inference or rare Flash access. For another, we fully utilize vertical stacking in LPDDR packaging. By having these extra Flash dies share the existing vertical stacks with the DRAM dies, we thus minimize the form factor overheads.

Lincoln Flash Die Architecture Overview. Figure 4(c) shows the overall architecture of our proposed Lincoln Flash die.

First, through shrinking the Flash array size, we enable up to 34GB/s internal read bandwidth per Lincoln Flash die, the considerations of which will be discussed in Section IV-B. Specifically, to improve the storage density, we utilize hybrid bonding, and splits the design into a Flash array layer and a logic layer that are vertically stacked. We also consider power supply issues for plane parallelism.

Second, to work with the LPDDR interface, we add a near-PHY SRAM-based global buffer in the logic layer. Due to the fast speed of SRAM and the near-PHY location [11] [108] [103], readout data from Flash can thus be temporarily staged in this buffer to serve the incoming LPDDR read requests without LPDDR timing violation. Besides, the buffer can also stage data input by LPDDR writes (e.g. data for program). Nevertheless, fetching data from the Flash layer to this buffer (i.e. Flash read operation) and MC-side ECC still exhibit high performance overheads. We resolve these issues in Section IV-C, without the need for redundant data movement between Flash, MC and DRAM as in conventional Flash designs.

Third, we place a simple near-Flash FP-Dot (dot product) logic within the Logic Block (LB) beneath every Flash plane, to efficiently support the matrix-vector multiply in the generation phase. Necessary ECC logic [102] is also added to ensure correctness. Moreover, we provide support for speculative decoding to further enhance the generation phase performance. Nevertheless, the frequent high-bandwidth Flash reads in the generation phase also incur serious endurance issues, especially read disturbs. When further considering the LPDDR-style 3D-stacked packaging, heat dissipation is also a concern. We will discuss hardware details of near-Flash logics in Section IV-D, and the endurance issues in Section IV-F. As for heat dissipation, our evaluation in Section V will show that the temperature increase is fully acceptable.

LLM Inference Workflow. The overall dataflow is depicted in Fig. 5. Lincoln system only relies on Flash dies for FC layer computation, with LPDDR for efficient NPU-side prefill and near-Flash logic for efficient generation, while computation of other layers are always carried out in the conventional way on the NPU. Thus, only the weight matrices for FC layers are stored in the Flash dies, while others (e.g. activations, KV cache and other weights) are stored in DRAM and used by the NPU as usual.

Essentially, FC layer computation involves GEMM and GEMV for prefill and generation respectively. To begin with, the matrix is sliced into multiple large partitions that can fit within the overall global buffer of all the Lincoln Flash dies at hand, and the computation will be carried out partition by partition. For GEMM computation by NPU in the prefill phase, the partition will be loaded into the buffer before being accessed and computed by the NPU. For GEMV by near-Flash logic in the generation phase, the input vector will be written to the global buffer by NPU and then broadcast to the near-Flash Logic Blocks, while Logic Blocks read out the matrix partition and carry out computation. The results will also be transmitted to the global buffer before readout by NPU. The NPU-side compute, NPU-Flash data transfers, and Flash-side data access and compute can be overlapped.

The related data layout and computation workflow will be discussed in Section IV-E. Since the weights in the Flash dies need to be accessed by both the NPU and near-Flash logic, we specifically discuss how to ensure access efficiency for both.

Lincoln Control System. Finally, for control and management over the Lincoln Flash and the newly introduced mechanisms, a corresponding software-hardware system stack design is required. Overall, we use SoC-side software/hardware to implement high-level management affairs, which sends transaction-level commands through LPDDR to Flash dies; and we rely on Flash-side controllers to carry out these commands within each Flash die. System details (e.g. command interface, instruction format, addressing, paging, etc.) will be discussed in Section IV-G.

B. Lincoln Flash Layer Design

Lincoln Flash layer is developed based on XL-Flash [50], which reaches a latency of 4us and incorporates 16 planes (with 4KB page) in one die. Concretely, we further shrink the plane size nearly by half along the bitline direction, and double the number of planes for more parallelism. This enables us to obtain an internal bandwidth of 34GB/s per Flash die. However, such a design will only worsen the two critical limitations already faced by original XL-Flash, i.e. density and current supply constraint [50]. In the following discussions, we show that these limitations can be resolved with available technologies.

Density. Generally, it is due to its planar design for Flash arrays and peripherals that XL-Flash already has a low area efficiency ($\sim$55%) [50]. Thus, we utilize the emerging but already commercialized hybrid bonding technology, which can hide peripheral area overhead and even improve peripheral performance. As shown in Figure 4(c), we put peripheral circuitry like page buffers that connect with Flash bitlines, as well as PHY, down to the logic layer, while leaving row decoders in the Flash array die. Since it is mainly these peripheral components that reduce area efficiency when further decreasing the array size, this already helps us reach an area efficiency of 74.6%, as illustrated in Table I in Section V-A, while leaving enough area in the logic layer for near-Flash logic.

As for viability of such a design, since small-array designs and Hybrid Bonding are already separately adopted for Flash products [49] [1], the remaining concern here is whether Hybrid Bonding adds significant latency to low-latency small arrays.

Fortunately, compared to bitline resistance and capacitance, the hybrid bonding interconnect incurs negligible parasitic effects ($>$1pF, $>10^5$Ohm vs $<$1fF, $\sim$1Ohm), as well validated in [38] [9] [10] [54]. By incorporating these effects in the simulator (as detailed in Section V-A), our evaluation shows that Hybrid Bonding contributes to negligible latency increase ($< 0.1\%$).

Power Supply. Flash dies that are compliant to commonly used Flash packages (e.g., BGA-132/152) have only a highly limited number of power pins (e.g., only 9 for Flash core operations). Supporting parallel all-plane operations for high data bandwidth on such dies would thus cause intolerable noise due to parasitic effects (e.g. ground noise) [50]. Besides, the circuits themselves also produce power noise. Therefore, standard-compliant designs have to limit the number of planes operated in parallel to mitigate noise if without latency increase [50] [44]. Fortunately, Lincoln already abandons such compliance for performance reasons, and thus can easily increase pins to tackle this issue. For conciseness, we discuss a straightforward design here (which can be further refined [85] [86] [30] [84] [55] [36]) — simply replicating the original power domains for Flash array operations, as well as the corresponding set of power/ground pins and related off-die decoupling capacitance, into multiple copies. Each copy, with equal supply capability and noise features as the original one, separately powers a limited number of parallel-operated planes same as that allowed in the original design. All together, these copies power all the planes, enabling them all to operate in parallel with noise/latency as low as in the original design, thus leading to much higher bandwidth. The increased pin count here is fewer than the latest 8-channel LPDDR design proposed by industry [77], and much fewer than the pin count support limit of BGA (700 and more) [39] [41] [40], thus being acceptable. Due to the use of hybrid bonding for peripheral hiding, as well as external high voltage supply (Vpp) that relaxes impedance requirement for power delivery, power peripheral and off-die capacitance overhead are also acceptable [50] [86].

C. Lincoln Logic Layer: LPDDR Interface

Basic Design. We can utilize the Flash-side near-PHY global SRAM buffer to meet the LPDDR timing requirements, as detailed below. **First**, before processing a partition of the weight matrix, the NPU memory controller (MC) issues a prefetch command by writing the command and the partition address to the Flash-side controller's command buffer. **Second**, the command triggers data reads from the Flash planes. The readout partition will be moved to the global buffer through the links between the Logic Blocks (LBs) and the buffer in Figure 4(c), and the address of the partition will also be recorded. Specifically, each plane has its own dedicated space in the buffer, thus no conflict issue exists. **Third**, on the NPU side, the MC polls the Flash-side status registers to see whether internal read-out has completed, and if so, it is then able to access data from the Flash-side global buffer with normal LPDDR access. Specifically, the fast speed of SRAM and the near-PHY location can guarantee that the strict LPDDR timings are always met [108] [103]. As for polling, it is done by accessing the recorded address of the buffered partition and comparing it with the expected address. Furthermore, the MC can use the tR parameters to accurately decide when to poll, thus reducing polling overheads. Yet, the long internal read-out latency still disturbs overall performance. **Fourth**, the ECC of the entire buffered partition needs to be carried out before starting computation on this partition, with the post-correction data stored back to the Flash-side global buffer or DRAM, similar to conventional Flash storage [66]. Unlike ECC for DRAM, such ECC for Flash features long latency (us) and is performed chunk-wise (1KB/chunk), which drastically mismatches the LPDDR latency ($\sim$100ns) and access size (32/64B). Thus it generally cannot be performed on demand during computation, but has to be performed entirely beforehand. Such ECC preprocessing is conventionally implemented out-of-Flash, e.g. in the NPU-side MC. It incurs redundant LPDDR accesses to the entire LLM weights and, bottlenecked by such accesses, results in significant runtime overheads.

Optimized Design. For optimization, we can utilize Double-Buffering-based Prefetch, as well as the near-Flash ECC modules [102] originally used for near-Flash computation, to overlap the Flash read latency and reduce the ECC costs.

Specifically, with Double Buffering, we can issue the Prefetch command for the next partition when doing computation on the current partition, thus overlapping the entire Flash-side prefetch process of the next partition with the NPU-side computation process on the current partition. Such an optimization is possible because the next partition to access is highly predictable for our target FC layer computation. Furthermore, the computation on a partition needs to access the entire partition through LPDDR, whose bandwidth is $>4\times$ lower than the Flash internal bandwidth. Thus the computation time is also at least $4\times$ higher than the Flash-side prefetch latency, and therefore the latter can be fully overlapped by the former.

As for ECC optimization, we utilize the near-Flash ECC modules [102], which are already introduced by our near-Flash logic design and placed beneath each Flash plane. By enabling the ECC to be carried out near Flash after data readout from each plane and before they are moved to the global buffer, the redundant LPDDR accesses required for NPU-side ECC are therefore eliminated, while the ECC execution is fully incorporated into the Flash-side prefetch process. Thus we can utilize the Double-Buffering-based Prefetch technique to hide the entire ECC latency. The details of near-Flash logics (e.g. ECC) will be discussed in the next subsection.

Besides, other optimizations also exist. For instance, while checking the recorded address of the buffered partition for polling, the NPU can opportunistically start reading the partition data, and abandon the read data when the check fails. Since the Prefetch optimization can easily hide the overall read latency, such opportunistic reads would seldom fail. Furthermore, since the SRAM read does not involve a long "activation" process as DRAM [11], we can minimize the row-activation-related LPDDR timing constraints for further performance improvement.

D. Lincoln Logic Layer: Near-Flash Logic

We place a dot product logic (FP-Dot) within the Logic Block (LB) beneath each Flash plane in the logic layer to support

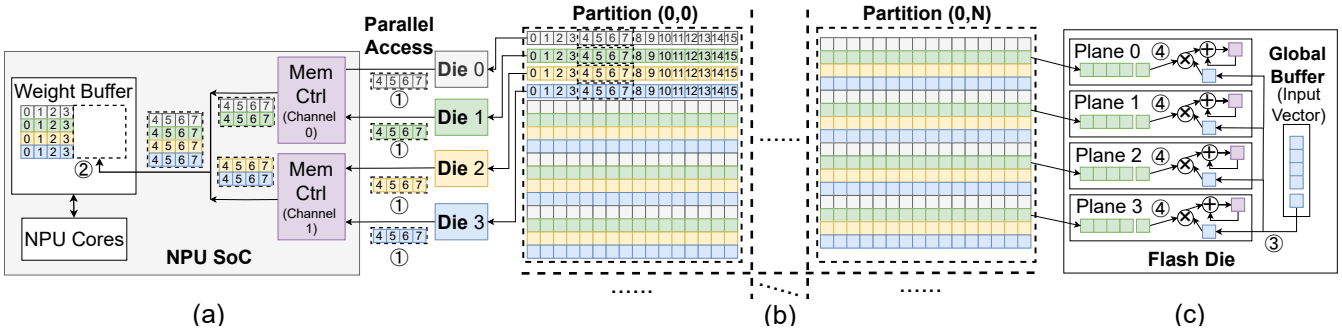

Fig. 6: Data layout and workflow: (a) round-robin row distribution across Lincoln Flash dies and tile access by NPU; (b) matrix partitioning for FC layer; (c) round-robin row distribution across planes within one Flash die and near-Flash computing flow.

GEMV computation. Considering the limited internal bandwidth of one plane (1.06GBps), we conclude that a two-element dot product logic is enough even when clocked at a conservative frequency of 400MHz, as illustrated in Figure 4(c). A small local register file is also added to maintain intermediate results. As for input and final output, they are buffered in the global buffer and transferred to/from Logic Blocks [30] [56] [52] through the bidirectional links shown in Figure 4(c).

As for ECC design, since our SLC-based design is generally more robust than more widely used TLC (Triple-Level Cell), we only need to use relatively simple BCH codes rather than much more complicated LDPC codes. Specifically, we adopt a very efficient design [102] that supports up to 64-bit error correction capability with small area and satisfactory throughput.

Speculative Decoding. For very large-sized LLMs (>100B), the design above is not enough to achieve real-time generation. Thus, we propose to further utilize speculative decoding [58], as briefly introduced below. Generally, for each generation iteration, we first use a small draft model (∼1B) to sequentially predict the next several (∼7) output tokens of our target large-sized model one by one. We then verify the prediction by executing the target LLM as with the append-style prefill, taking these predicted tokens as input and processing them in batch, while accessing the target model weights only once. The first several tokens that are verified to be true are accepted as the true outputs. In this way, for each iteration, multiple tokens can be generated. Since the draft model is often smaller than the target model by almost 100×, the per-token generation latency, as well as memory traffic and memory energy consumption, can thus be reduced while with no harm to model quality, as long as >1 token is accepted by verification [63].

To support this technique, we increase the near-Flash computing logic for such batched computation — the FPDot unit is replicated, and the register file is enlarged. Yet, as shown in Table I, this overhead is still small enough compared to the overall area.

E. Data Layout and Workflow

The tiling and scheduling schemes of near-Flash computing and NPU execution for our target FC layers generally differ, and it is not preferable to maintain two copies for each. Specifically, only 265GB space is available in our design if with 2 LPDDR packages. If we keep two copies, Lincoln would be unable to serve >71B model, which is highly constrained. Taking this concern into account, we design the data layout and workflow in Figure 6, as discussed below. For conciseness, we mainly focus on row-major matrix layout support for the NPU, with column-major discussed at the end of this subsection.

Data Layout. First, we slice the matrix into large partitions that can fit in the aggregate global buffer size of all the Flash dies, as shown in Figure 6(b). This facilitates the use of buffering and prefetch techniques. Second, as shown in Figure 6(a), the rows in the partition are distributed to each Flash die in a round-robin fashion. Such coarse-grained row-wise distribution facilitates near-Flash implementation, compared to conventional fine-grained data interleaving. Besides, such distribution is also sufficient for the NPU to fully utilize LPDDR channel parallelism for efficient matrix tile access, as will be discussed later. Third, as shown in Figure 6(c), the rows located in one Flash die are further distributed to each plane in a round-robin fashion, and sequentially stored within one (or more) page. The page address allocated in each plane is identical, so as to meet the multi-plane operation requirement [92].

Near-Flash Computing Flow. Near-Flash computing on a FC layer is carried out partition by partition, and row by row within each partition. First, before computation, the NPU broadcasts the input vectors to the global buffers of each Flash die. Second, for each Flash plane, the Flash page corresponding to the partition rows is read out and corrected with the ECC engine. Third, as illustrated in Figure 6(c), the controller sequentially broadcasts the input vector to each plane (③). On receiving, the local near-Flash computing unit in each plane carries out dot product between the received vector elements and the corresponding matrix elements, with the intermediate results stored in local register file (④). Fourth, the final results will be moved to the global buffer before read out by the NPU. Specifically, to maximize performance, the Flash page readout and computation processes are pipelined. The input vector write, near-Flash read/compute, and result transfer processes are also overlapped across partitions.

Efficient NPU Execution. NPU's computation on a FC layer is also carried out partition by partition. Before computation, the data will be first prefetched to the global buffer from Flash planes as discussed before, which is overlapped with the computation on the previous partition. During computation, the NPU typically accesses partition data tile by tile, and an access to a tile is

illustrated in Figure 6(a), which exhibits the following three features. First, since a tile typically spans many rows, and since we distribute rows across dies in a round-robin fashion, the NPU can access the rows of a tile from corresponding dies in parallel (①), thus fully utilizing LPDDR's channel parallelism and the overall LPDDR bandwidth. Second, by adjusting readout data length from the channels for a tile (①), and by concatenating multiple reads (②) or splitting one read within the weight buffer, the NPU is able to flexibly adjust the size of the accessed tile. Third, since the whole partition is already prefetched to the Flash-side global SRAM buffer, which, by the nature of SRAM, supports random and uniformly speedy access to any data within the buffer, the NPU can thus access the tiles within the partition in any order at will without any risk of performance loss. All in all, with flexible tile sizes and access orders, the NPU can therefore flexibly implement tiling and scheduling strategies that best meet its demand. Together with fully utilized LPDDR bandwidth, the efficiency of NPU-side computation is thus ensured.

Supporting Column-major for NPU. This can be implemented by changing distribution scheme across dies to column-major, while with distribution within die across planes kept as row-major. This row-major layout can be easily transformed to column-major when data are readout from planes and moved to the global buffer, by adjusting the order and destination addresses of data movement. Such a design facilitates near-Flash computing, since dataflow within Flash die can now be kept unchanged, only that input and output vectors, unlike row-major scheme, need to be split and accumulated across dies instead.

F. Endurance

Read disturbance is an important concern for Lincoln. Generally, LLM weights do not need frequent update, but involve frequent reads. Even if optimistically assuming our SLC Flash can endure up to 10K read cycles[1] before the raw error bit rate exceeds ECC capability, which is equivalent to 10K LLM execution iterations, such a threshold can be easily exceeded within 2 hours if we keep requesting the LLM to generate long outputs.

Fortunately, we find that read reclaim by re-programming them to Flash suffices to resolve this problem, which is similar to data refresh for retention error mitigation [14] [12]. First, Program/Erase (P/E) can generally reset the accumulated read disturbance error [73] [74] [13]. Second, unlike MLC/TLC, SLC can endure up to 100K P/E cycles [49], while the infrequently updated LLM weights leave most of this budget unused. This endurance threshold can be further boosted to at least 4M P/E cycles, if we shorten the retention time requirement from 1 year to 3 days, by enforcing thorough data refresh once every 3 days. Such refresh itself consumes negligible P/E cycles (609 cycles per 5 years) [48]. Third, the endurable P/E cycles of all Flash cells can be well utilized through wear leveling. For relatively static data like weights, good wear leveling is easy to achieve, e.g., moving data on reclaim/refresh and cycling through the whole available storage space [48]. With these, we can easily mitigate read disturbs while keeping P/E cycle consumption well below the endurance threshold. Specifically, if we target a warranty of 5 years and conservatively assume the read disturb threshold as 5K read cycles, then for our evaluation in Section V, the maximum P/E cycle consumption required is merely 310K even if we keep running the LLM generation phase all the time. Indeed, the consumption stays well below 4M even with lower read disturb thresholds (down to $<$1K).

Finally, to avoid performance disturbance caused by batched read reclaims (or data refresh), we can distribute them over every execution iteration [44] (or devices' idle times), e.g. reclaim 1/5K of the model per iteration, which causes negligible execution time overhead ($\sim$0.4% with program latency of 75us [50]).

[1]Although statistical research on read disturbance of 3D SLC NAND Flash is rare, we find that MLC/TLC can endure up to 5K-8K read cycles [73] [74] [13]. Generally, regular NAND Flash products can endure 1K$\sim$10K read cycles.

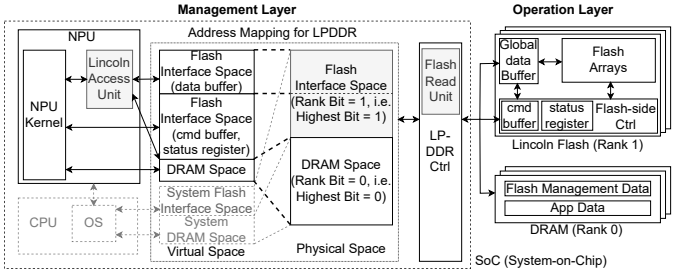

Fig. 7: Lincoln Control System Design Overview

G. Lincoln Control System

Fig. 7 shows the system design for control of previously discussed functions. Management Layer (ML) implements high-level management logic (e.g. address translation, wear leveling, read reclaim, etc.), and issues transaction-level commands to Operation Layer (OL), while OL breaks down received commands into micro-operations and carry them out [45] [51] [81]. OL is implemented with the Flash-side Controller (FC), while ML is implemented with software and supportive hardware on SoC side.

1) ML-OL Interaction Interface: ML issues transaction-level commands to Flash-side controller by writes to Flash-side command buffer, checks command execution status by reads from Flash-side status registers, and prepares/fetches input/output data of transactions by writes/reads to/from Flash-side global data buffer. These Flash-side buffers/registers are placed at PHY-side and memory-mapped, so that they can be accessed through LPDDR, while compliant with LPDDR timing, similar to [108] [103]. ML software guarantees that no conflict occurs [53] for SoC-side/Flash-side access to global data buffer through status checks. And a Flash-side counter records the receive of new commands.

Specifically, transaction-level command is in 64 bits, with a 16-bit OP field to specify functions (PageRead / PageProgram / Erase / ComputePartition / MoveResult, etc.), a 16-bit BUFF_ADDR field to specify the involved Flash-side global data buffer address, and a 32-bit FLASH_ADDR field to specify the involved Flash memory address or registers in plane-side compute blocks. The command supports both single- and all-plane operation. For the latter, same local address bits are used for each plane to fit the 64-bit command. We also use batched commands and padding for compatibility with LPDDR access granularity.

Besides, for ease of management, we change the LPDDR physical address mapping and put the only one rank bit of LPDDR at the highest bit. In this way, Flash interface address space and DRAM address space are clearly separated, each being contiguous and not interleaved with each other. Thus DRAM space can be naturally managed and accessed as before, with no complexity increase for OS paging or non-inference applications.

2) Flash-side Controller (FC): FC performs transaction-level commands with a state machine to appropriately handle the sequence and timing of decomposed micro-operations. While similar to conventional controllers [45] [81], it also has differences in states and micro-operation chains to account for Lincoln's specific architecture and new functions (e.g. specific instruction format, near-Flash compute, etc.). We estimate that FC's area overhead is negligible (<%1 of total area) [81] [87].

3) ML Software: If LLM inference is not running, OS takes charge of all Lincoln Flash management affairs. The related data structures for management (e.g. Flash address translation) are stored in DRAM. If LLM inference is running, the NPU kernel takes full control to avoid system call penalty, by mapping the physical address spaces of Flash interface and management data structures to its own virtual address space through OS. With these, it can perform Flash memory address translation / Flash read / read reclaim / wear leveling / garbage collection all on its own. For ease of management, we contiguously map physical space of Flash interface to software's virtual address space. We also disable original LPDDR channel interleaving and bypass cache [5] [88] for Flash interface access. To resolve performance issues, major operations involved in inference are accelerated as below.

4) Flash Access Acceleration: We add Lincoln Access Units (LAU) in NPU to enable efficient access to Flash-side global data buffer, with patterns discussed in Section IV-E. LAUs interface to NPU kernel through instructions, similar to H100's TMA [18] [70]. They may also be integrated with existing TMA-alike units (if exist). For Flash read transactions, we add Flash Read Unit in LPDDR controller to efficiently handle interaction with OL, as discussed in Section IV-C. NPU kernel interacts with it indirectly through LAU. LAU also handles Flash memory address translation, supporting efficient round-robin address mapping [48], as well as more general partition-granularity table-based mapping (with prefetch buffers of 16KB in total), as discussed in Section IV-F. Both incur <0.1% runtime overhead.

V. Evaluation

In this section, we try to answer the following questions:

- How much is the area/power cost of Lincoln?
- How much performance and energy improvement does Lincoln make compared to conventional SSD-based systems? Does Lincoln satisfy the real-time latency requirement?
- How do our proposed techniques contribute to the performance and energy improvement?
- Are heat dissipation and cost of Lincoln acceptable?

A. Methodology and Area/Power/Latency Estimation

Baseline and Configuration. To answer the questions above, we evaluate Lincoln, a baseline (Base) that uses conventional Flash drive, and an enhanced version of Base with speculative decoding (BaseS). The configurations of the NPU [29], the memory system, and the storage system are summarized in Table I, including estimated area/power/latency results. For fair comparison, we adopt identical configurations for Lincoln and the baselines, except that the LPDDR packages of Lincoln are equipped with our proposed Lincoln Flash dies, while the baselines have to use NVMe-based SSD. We also evaluate a conservative 2-LPDDR-package and an aggressive 4-package configuration for Lincoln and baselines, abbreviated as Lincoln-2 / Base-2 / BaseS-2 and Lincoln-4 / Base-4 / BaseS-4. Considering that vendors are now pushing >100 TOPS SoC to consumer market [35], the 32 TOPS configuration is actually a conservative approximation to real products.

TABLE I: Simulation Configuration

NPU Configuration	
Compute	32 TOPS (BF16 Tensor Core), 4.60 W
SRAM	4MB/2MB Output/Input buffers, 0.431 W
Memory System Configuration	
LPDDR5X-8533, 16Gb, x8, FR-FCFS; $\sim$0.3Gb/mm^2 (for DRAM) 2 / 4 Packages $\times$ 4 Channels $\times$ 2 DRAM dies (+ 2 Lincoln Flash dies)	
Lincoln Flash Configuration	
Flash layer	SLC, 96 wordline-layers, (4KB + 448B ECC) / page, 768 pages / block, 177 blocks / plane, 32 planes 4.1484Gb per plane, 132.75Gb per die tR = 3.426 us, Energy = 98.0 nJ / page read Plane area = 1.702 mm^2 Per-plane peripheral (e.g. row decoder) area = 0.579 mm^2 Total area = 72.99 mm^2, 1.8Gb/mm^2
Logic layer	16 FMACs + 8KB register file per plane 0.0708 mm^2, 6.97 mW per plane ECC: BCH(9088, 8192, 64), 14.6Gb/s per plane 0.1478 mm^2, 5.24mW per plane [102] 260 KB Global buffer 0.4095 mm^2, 18.4 mW in total Per-plane peripheral (e.g. page buffer) area = 1.67 mm^2 Other Peripheral area = 10.17 mm^2 in total Total area = 71.0 mm^2, near-Flash logic power = 409.1 mW
Storage Configuration	
TLC, 2TB total capacity, tR = 56us; $\sim$10Gb/mm^2 8 channels $\times$ 4 dies $\times$ 4 planes $\times$ 683 blocks $\times$ 1536 pages $\times$ 16KB NVMe, PCIe 4.0, 4 lanes, 8GB/s external bandwidth, 1.2 GBps/channel;	

For the latency and energy estimation of the Flash arrays in Lincoln, we utilize the NVSim-based [21] 3DFPIM simulator [54], using publicly available parameters from research community and industrial reports [110] [62] [32] [64], and validate against the results reported by [54]. Specifically, considering that NVSim has assumed a current sensing latency different from that of mainstream Flash products [65] [66], and that low-latency Flash designs like XL-Flash [50] also have their own unique considerations, we have compensated the simulator with data reported in [50]. Besides, parasitic resistance/capacitance of hybrid bonding [38] [9] [10] is also considered, which contributes to <0.1% latency increase. In the end, we derive $tR = 3.426us$ for Lincoln's read latency estimation. Considering that XL-Flash's array is larger than Lincoln's array (by almost 2×), and thus that its latency ($tR = 4us$) also upper-bounds Lincoln's [50], we can safely conclude that Lincoln's real performance be at most 17% worse than our simulation.

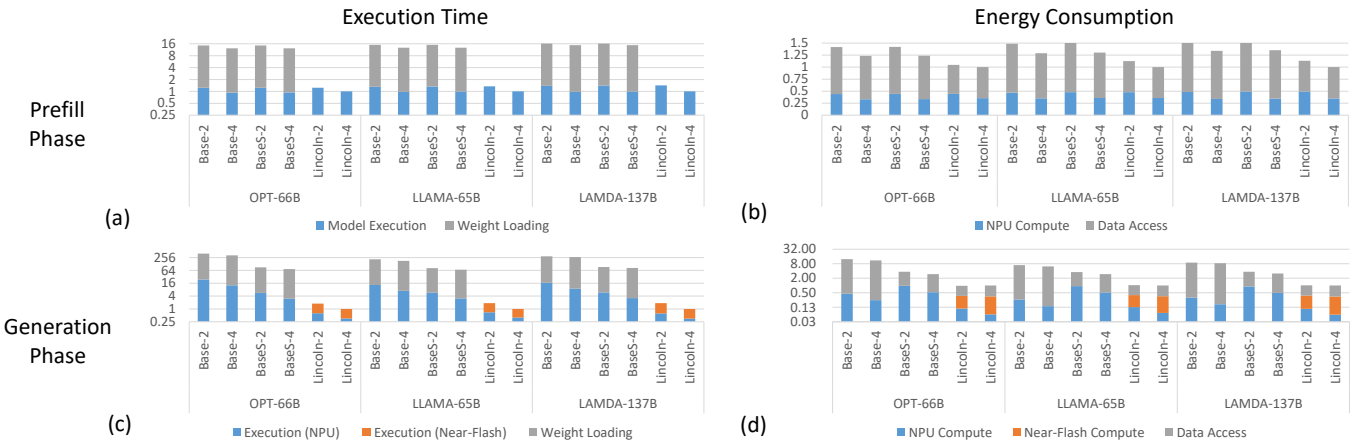

Fig. 8: Execution time and energy consumption comparison. For each model, execution time and energy data are all normalized to the corresponding overall execution time and energy of Lincoln-4. The results for generation phase are shown in logarithmic scale.

Furthermore, we estimate the area of Flash arrays and the peripheral overheads based on the data and die photo of industrial implementation [50] for accuracy, and with LPDDR PHY [16] incorporated. Peripheral replication (mainly charge pumps) for power supply is also considered, occupying approximately 7% of the overall area.

Finally, we derive the area and power of compute units (including ECC) from previous works [102] [80], and estimate those of on-chip SRAM buffers based on Destiny [79]. Such works for both NPU and near-Flash logic, since Hybrid Bonding enables logic die to use mature CMOS technology without influence by Flash-array manufacture process. The data are scaled down to 7nm for NPU and to 20nm for Near-Flash logics with scaling factors reported in [97] [87]. And the near-Flash logic area is further multiplied by 2× as with previous work [99], considering that fewer metal layers are preferably used for logic layer in memory products than for normal CMOS chips so as to reduce costs, which results in area increase. All in all, we observe that near-Flash logic occupies a small area percentage, 7.98%, and that a Flash area efficiency of 74.6% is achieved with hybrid-bonding-based 3D stacking, much better than that of XL-Flash (∼55%).

Performance/Energy/Heat-Dissipation Modeling. We simulate the performance of NPU based on ONNXim [29], a cycle-accurate simulator that has integrated Ramulator 2 [60] for DRAM and LPDDR interface simulation. We further combine it with modeling for our proposed Lincoln Flash memory, based on conventional NAND Flash's die model [45]. We integrate formulas from DRAMPower [15] into Ramulator 2 to estimate LPDDR5X energy consumption [67]. We utilize MQSim [89] to simulate NVMe-based SSD behavior, and add support for energy consumption estimation, similar to SimpleSSD [46] [25]. The energy data of Flash arrays are derived from the NVSim-based 3DFPIM simulator, which is accurate on array level [54]. While it may have inaccuracy for peripheral's power-supply efficiency, such inaccuracy is offset for relative energy reduction simulations by applying it to both Lincoln and baselines. Finally, we analyze the heat dissipation of Lincoln in LPDDR-style vertical packaging based on HotSpot [105]. For conservative estimation, we aggressively consider possible energy consumption deviations from simulator's reports, as detailed in Section V-D.

Benchmarks. We use OPT-66B [106], LLaMA-65B [94], and LAMDA-137B [93] as our target models. For speculative decoding, we use OPT-125M [100], NoFT-796M [100], and LAMDA-2B [58] as their draft models respectively, and use the worst acceptance rates reported in [58] [100]. We test performance on the commonly used ShareGPT dataset [2] as previous works [107] [31], while targeting a strict 90% tail latency.

B. Performance Comparison

Leveraging internal-bandwidth-boosted, LPDDR-interfaced and compute-enabled Flash memory, Lincoln achieves remarkable performance increase and energy reduction over the speed-constraint SSD-based baseline, and reaches readl-time latency.

First, for prefill phase latency in Fig. 8(a), Lincoln-2 and Lincoln-4 exhibit 11.44×/11.46× and 13.23×/13.24× speedups over Base-2/BaseS-2 and Base-4/BaseS-4, utilizing NPU and LPDDR-interfaced high-speed Flash memory. Specifically, as illustrated by the execution time breakdown, Lincoln's LPDDR-interfaced design entirely removes the SSD-based weight loading procedure that dominates Base's overall latency. Furthermore, by well utilizing the boosted Flash internal bandwidth and fully excavating the performance potential of LPDDR links, it enables NPU to work with Flash dies as efficiently as with DRAM dies, thus achieving overall latency almost identical to Base's model execution time, with only small increase due to extra draft model prefill computation. BaseS also sees such draft model overheads, which are insignificant compared to the high overheads of weight loading. Besides, it is also interesting to note that Lincoln-4's relative speedup over Lincoln-2 is much lower than its available bandwidth increase. This is because the overall performance is now bottlenecked by compute capability rather than data access.

Second, for prefill phase energy in Fig. 8(b), Lincoln-2 and Lincoln-4 shows 34.5%/35.6% and 30.1%/31.2% reduction over Base-2 and Base-4 on average. The reduction is due to the elimination of redundant DRAM writes involved in SSD-based weight loading procedure, and due to Flash array shrinking which decreases the Flash read energy consumption for weights.

TABLE II: Absolute prefill/generation latency of Lincoln.

	Prefill Latency		Generation Latency	
	Lincoln-2	Lincoln-4	Lincoln-2	Lincoln-4
OPT-66B	1.633s	1.315s	0.085s	0.046s
LLaMA-65B	1.700s	1.269s	0.163s	0.087s
LAMDA-137B	3.772s	2.667s	0.264s	0.142s

The reduction percentage decrease of Lincoln-4 over Lincoln-2 mainly lies in that the weight-related overheads for reduction are slightly decreased when #packages is increased, since more DRAM space is now available to pin more weights in memory and prevent them from repeated loading from Flash. Interestingly, this effect does not lead to the decrease of Lincoln's speedup over baselines in Fig. 8(a), since doubled #packages results in more significant reduction of Lincoln's execution time (1.33×) than time reduction of weight loading from Flash (1.13× for Base).

Third, for generation latency per token in Fig. 8(c), Lincoln-2 and Lincoln-4 realize 158.4× and 254.1× speedups over Base-2 and Base-4 on average, by leveraging both near-Flash logic and speculative decoding to fully utilize boosted Flash internal bandwidth and enable multi-token generation per iteration respectively. It also achieves 47.8× and 76.6× speedups over BaseS-2 and BaseS-4 on average, with only the advantage of Lincoln's near-Flash compute. Compared to the prefill phase, Lincoln-4's performance increases more significantly over Lincoln-2, since the generation phase is highly memory-intensive, and thus the doubled bandwidth exerts higher influence on performance.

Fourth, for generation energy per token in Fig. 8(d), Lincoln-2 and Lincoln-4 realize 9.11×/3.66× and 8.46×/3.10× reduction over Base-2/BaseS-2 and Base-4/BaseS-4 on average. Compared to the reduction in the prefill phase, the boost over BaseS lies in the use of near-Flash computing that eliminates LPDDR access to LLM weights, and the further boost over Base lies in speculative decoding that enables one target-model weight access to generate multiple tokens instead of one. Yet, in contrast to overall reduction, the energy fraction for compute of Lincoln and BaseS actually increases compared to Base on average, since speculative decoding introduces extra and redundant compute works. Besides, near-Flash logic uses less advanced technology node than the NPU, while BaseS has to inefficiently use compute-rich tensor units to deal with the small-batched compute in verification, which generally only verifies a small draft sequence each time to avoid waste caused by limited acceptance rate.

Fifth, we note that Lincoln-2, though featuring only 265.5GB LPDDR-interfaced Flash, is enabled to serve LAMDA-137B as efficiently as with smaller models for both prefill and generation phases. This is due to our data layout and workflow design that requires only one weight copy for efficient access by both the NPU and near-Flash logic.

Finally, Table II summarizes the prefill and generation phase latency of Lincoln. In comparison, previous system works [107] have taken 4.0s/0.2s as acceptable prefill/generation latency target for large-sized LLMs, while industry implementation [23] also uses 5.0s as acceptable prefill latency. With these, we can safely conclude that Lincoln-2/Lincoln-4 has almost/fully met all the real-time latency targets.

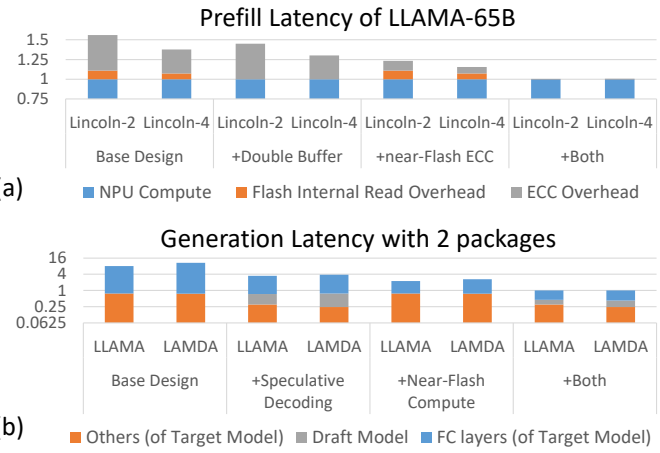

Fig. 9: Ablation study for techniques on (a) prefill and (b) generation. All latency normalized to fully optimized design.

C. Ablation Study

In this subsection, we study how our proposed techniques contribute to aforementioned results. For the prefill phase, we discuss LLaMA-65B (LLaMA) under both the 2- and 4-package configurations, as shown in Fig. 9(a). The basic design here refers to that described in Section IV-C. For the generation phase, we discuss both LLaMA and LAMDA-137B (LAMDA) under the 2-package configuration, as shown in Fig. 9(b). The basic design utilizes the NPU and LPDDR interface for compute and data access, similar to the prefill phase.

For the prefill phase, we make the following observations. First, Flash read latency and ECC collectively cause high overheads (i.e. 56.0% and 37.6%) for Lincoln-2 and Lincoln-4, when double buffering and near-Flash ECC are disabled. The overheads here are dominated by ECC, since ECC is bottlenecked by LPDDR interface, while internal Flash read bandwidth of Lincoln is much faster than LPDDR. Second, although capable of fully hiding Flash-side read latency, double buffering alone achieves very slight performance improvement (10.98%/7.36%), because the overhead-dominating ECC is implemented on the memory controller side and cannot be reduced at all. Third, near-Flash ECC alone reduces ECC costs by moving ECC to the Flash side and thus removing the LPDDR bottleneck, but the remaining ECC and Flash read overheads, however, are still as large as 23.2% and 15.6%. Fourth, when with both optimizations combined, the overheads can eventually be almost fully eliminated, since double buffering can hide the entire Flash-side latency, including both internal reads and Flash-side ECC. Finally, we observe that the overheads for Lincoln-4 are always lower than those for Lincoln-2. This is because Lincoln-4 is more compute-bound than Lincoln-2, and thus the NPU compute time proportion is relatively increased, with the overhead proportion relatively decreased.

As for the generation phase, first, speculative decoding alone boosts the execution time of both FC layers and other layers by 2.61× and 3.226× for LLaMA-65B and LAMDA-137B. Yet, it also introduces draft model overheads (14.0% and 15.9%). The higher overhead for LAMDA is due to the much larger size of its draft model. Second, near-Flash computing alone realizes

speedups of 3.60× and 4.05×. It reduces FC layer execution time, but leaves other costs (mainly attention calculation over KV Cache) unchanged, thus much increasing the proportion of the other costs. Indeed, the lower speedup here of LLAMA actually roots in its relatively larger attention calculation time over FC layer computation time. Third, when with both optimizations enabled, 7.99× and 10.63× speedups can be achieved overall. Specifically, near-Flash computing is also used for draft model execution, boosting the performance of NoFT-796M and LAMDA-2B by 2.938× and 2.935× respectively.

D. Heat Dissipation Analysis

With all low-latency Flash planes and near-Flash logic working in parallel, and with all these full-speed running Flash dies stacked vertically in LPDDR-style packages, heat dissipation might be a problem. Moreover, Flash array energy consumption of real implementation may deviate from the analytic results of simulation. Thus, we perform a pressure test for heat dissipation analysis, with the Flash array energy consumption respectively set as 1×, 1.5×, 2×, 2.5× and 3× of the simulated result. Fortunately, the reported highest temperature increase corresponding to these settings are 3.0K, 3.9K, 4.8K, 5.7K and 6.6K, which are either mild or well acceptable. Thus we can safely conclude that the heat dissipation of Lincoln is fully acceptable.

E. Cost Analysis

Since we reuse existing LPDDR interface and utilizes unused area in peripheral die for near-Flash logic, Lincoln's cost mainly comes from SLC Flash, hybrid bonding (HB) and peripheral die. To account for them, we base our analysis on YTMC's 128-layer TLC Flash ($8.5Gb/mm^2$) [91], which already incorporates hybrid bonding and peripheral die. Its cost is \$0.11/GB. For normal/conservative estimation, we take its yield as 80%/50%.

With SLC and $1.8Gb/mm^2$ density, Lincoln's cost mounts to \$0.52/GB. Further considering Lincoln's 1.22× area and 2× HB I/O count per die for page buffers, the yield can be conservatively estimated as $(80\%)^{(1.22\times2)} = 58\%$ or $(50\%)^{(1.22\times2)} = 18\%$, and the final cost rises to \$0.72/GB or \$1.44/GB. For 2-package configuration, \$191/\$382 is required with 5-year warranty.

For commercial comparison, a MacBook with 36GB DRAM requires ∼\$3000 [8], while ChatGPT with unlimited access requires \$20/month [71] and \$1200 for 5 years. For technical comparison, consider a system with conventional Flash design that generally features 1∼2GBps internal bandwidth per die [90]. Whether it features near-storage processing [37] [47] or not, 200∼500 Flash dies are required to reach internal bandwidth equal to 2-package Lincoln's, which generally requires $\$1500 \sim \$5000$ [90]. Thus Lincoln is sufficiently cost-competitive.

VI. Related Works

Many algorithmic techniques are under study for LLM deployment on the edge, like compression (quantization / pruning / distillation) [24] [95] [22] [4] [61] [98] [27] [28] or training of stronger small models [22] [4] [28]. Besides, recent works like LLM-in-a-Flash [7] also use dynamic sparsity to host models larger than available DRAM-capacity with SSD. Generally, Lincoln-style design does not actually contradict these techniques. First, Lincoln can host larger models when combined with compression, like 4-bit 405B model. Second, Lincoln-style designs (e.g. a half-sized Lincoln package) can enable much smaller devices (like smart phones) to host small models without aggressive compression, or host compressed larger models, like 16/8-bit 7B/13B or 4-bit 70B/100B models. Third, Lincoln may also utilize dynamic sparsity for better performance, which we defer to future work.

Previous near-memory/storage systems generally do not target the edge scenario in our concern, which requires high capacity/speed but only allows limited SoC I/O and a limited number of memory dies. Specifically, near-storage proposals [37] [57] [47] mostly use conventional Flash dies and rely on conventional SSD systems, which, as already discussed, feature undesirably low internal/external bandwidth within limits of consumer devices. Besides, they mostly target operations different from Lincoln's. Smart-Infinity [37], for instance, targets weight update in AI training. While it uses gradient compression to mitigate external bandwidth limit, such only applies to weight update. As for low-density DRAM systems (e.g. DIMM-based [108]/near-bank [75] designs), they generally cannot provide sufficient capacity. The LPDDR-based CXL-PNM [77], for instance, requires extra 8-package/64-channel/256-die DRAM for high capacity, infeasible for most edge devices. Besides, it uses CXL-controller-based near-memory unit for whole inference. This is infeasible for Lincoln, whose weak near-Flash logic can hardly reach our real-time goal for prefill (>5s for all benchmarks on Lincoln-2), and cannot perform attention due to Flash endurance issues. Instead, Lincoln uses existing central SoC for intensive compute as well as attention, and its existing LPDDR I/O to fetch weights from Flash.

Finally, though DRAM-LPDDR has been used for direct link with NVM dies [26] [43] [42], it is mainly for NVM that features faster read and more light-weight ECC, like NOR Flash [26] [34] [68] [33] [109], rather than NAND Flash. This is because NAND Flash's high read latency and more complex controller-side ECC cause high performance penalty, and due to design complexities like endurance management issues. Lincoln is the first to both explore this direction for LLM acceleration and well resolve related issues.

VII. Conclusions

This paper presents Lincoln, a device-architecture co-design that resolves the Flash storage bottleneck for large-sized LLM execution on consumer devices with LPDDR-Interfaced, Compute-Enabled Flash memory. Evaluation shows that Lincoln enables real-time 50∼100B LLM Inference.

Acknowledgements

We thank the shepherd and the anonymous reviewers for their insightful feedbacks. We also thank Hunjun Lee and Weihong Xu for helping us understand and set up the 3DFPIM simulator, and Zongliang Huo and Lei Jin for detailed discussion on 3D NAND Flash. This work was supported in part by No.31513010104, and in part by the National Natural Science Foundation of China (Grant No. U22B2024).

REFERENCES

[1] "About xtacking - ymtc," accessed on July 1st, 2024. [Online]. Available: https://www.ymtc.com/en/technicalintroduction.html

[2] "Sharegpt." [Online]. Available: https://sharegpt.com/w

[3] "Ulltradimm ssd solution brief: Flash storage on the memory bus." [Online]. Available: https://www.sandisk.com/content/dam/sandisk-main/en_us/assets/resources/enterprise/white-papers/sandisk-solution-brief-ulltradimm-on-the-memory-bus.pdf

[4] M. Abdin, J. Aneja, H. Awadalla, A. Awadallah, A. A. Awan, N. Bach, A. Bahree, A. Bakhtiari, J. Bao, H. Behl, A. Benhaim, M. Bilenko, J. Bjorck, S. Bubeck, M. Cai, Q. Cai, V. Chaudhary, D. Chen, D. Chen, W. Chen, Y.-C. Chen, Y.-L. Chen, H. Cheng, P. Chopra, X. Dai, M. Dixon, R. Eldan, V. Fragoso, J. Gao, M. Gao, M. Gao, A. Garg, A. D. Giorno, A. Goswami, S. Gunasekar, E. Haider, J. Hao, R. J. Hewett, W. Hu, J. Huynh, D. Iter, S. A. Jacobs, M. Javaheripi, X. Jin, N. Karampatziakis, P. Kauffmann, M. Khademi, D. Kim, Y. J. Kim, L. Kurilenko, J. R. Lee, Y. T. Lee, Y. Li, Y. Li, C. Liang, L. Liden, X. Lin, Z. Lin, C. Liu, L. Liu, M. Liu, W. Liu, X. Liu, C. Luo, P. Madan, A. Mahmoudzadeh, D. Majercak, M. Mazzola, C. C. T. Mendes, A. Mitra, H. Modi, A. Nguyen, B. Norick, B. Patra, D. Perez-Becker, T. Portet, R. Pryzant, H. Qin, M. Radmilac, L. Ren, G. de Rosa, C. Rosset, S. Roy, O. Ruwase, O. Saarikivi, A. Saied, A. Salim, M. Santacroce, S. Shah, N. Shang, H. Sharma, Y. Shen, S. Shukla, X. Song, M. Tanaka, A. Tupini, P. Vaddamanu, C. Wang, G. Wang, L. Wang, S. Wang, X. Wang, Y. Wang, R. Ward, W. Wen, P. Witte, H. Wu, X. Wu, M. Wyatt, B. Xiao, C. Xu, J. Xu, W. Xu, J. Xue, S. Yadav, F. Yang, J. Yang, Y. Yang, Z. Yang, D. Yu, L. Yuan, C. Zhang, C. Zhang, J. Zhang, L. L. Zhang, Y. Zhang, Y. Zhang, Y. Zhang, and X. Zhou, "Phi-3 technical report: A highly capable language model locally on your phone," 2024. [Online]. Available: https://arxiv.org/abs/2404.14219

[5] M. Alian, S. W. Min, H. Asgharimoghaddam, A. Dhar, D. K. Wang, T. Roewer, A. McPadden, O. O'Halloran, D. Chen, J. Xiong, D. Kim, W.-m. Hwu, and N. S. Kim, "Application-transparent near-memory processing architecture with memory channel network," in Proceedings of the 51st Annual IEEE/ACM International Symposium on Microarchitecture, ser. MICRO-51. IEEE Press, 2018, p. 802–814. [Online]. Available: https://doi.org/10.1109/MICRO.2018.00070

[6] K. Alizadeh, I. Mirzadeh, D. Belenko, K. Khatamifard, M. Cho, C. C. D. Mundo, M. Rastegari, and M. Farajtabar, "Llm in a flash: Efficient large language model inference with limited memory," 2024. [Online]. Available: https://arxiv.org/abs/2312.11514

[7] K. Alizadeh, S. I. Mirzadeh, D. Belenko, S. Khatamifard, M. Cho, C. C. Del Mundo, M. Rastegari, and M. Farajtabar, "LLM in a flash: Efficient large language model inference with limited memory," in Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), L.-W. Ku, A. Martins, and V. Srikumar, Eds. Bangkok, Thailand: Association for Computational Linguistics, Aug. 2024, pp. 12 562–12 584. [Online]. Available: https://aclanthology.org/2024.acl-long.678

[8] Apple, "Customize your 14-inch macbook pro - space black," accessed on October 13th, 2024. [Online]. Available: https://www.apple.com/shop/buy-mac/macbook-pro/14-inch-space-black-apple-m3-max-with-14-core-cpu-and-30-core-gpu-36gb-memory-1tb

[9] B. Ayoub, S. Lhostis, S. Moreau, E. L. Perez, J. Jourdon, P. Lamontagne, E. Deloffre, S. Mermoz, C. de Buttet, V. Balan, C. Euvard, Y. Exbrayat, and H. Frémont, "Impact of process variations on the capacitance and electrical resistance down to 1.44 μm hybrid bonding interconnects," in 2020 IEEE 22nd Electronics Packaging Technology Conference (EPTC), 2020, pp. 453–458.

[10] B. Ayoub, S. Lhostis, S. Moreau, E. Souchier, E. Deloffre, S. Mermoz, M. G. G. Cacho, N. Szekely, C. Rey, E. Aybeke, V. Gredy, P. Lamontagne, O. Thomas, and H. Frémont, "Sub 1 μm pitch achievement for cu/sio2 hybrid bonding," in 2022 IEEE 24th Electronics Packaging Technology Conference (EPTC), 2022, pp. 418–424.

[11] R. Balasubramonian, E. J. Natalie, and M. Martonosi, Innovations in the Memory System. Morgan & Claypool Publishers, 2019.

[12] Y. Cai, Y. Luo, S. Ghose, E. F. Haratsch, K. Mai, and O. Mutlu, "Read disturb errors in mlc nand flash memory," 2018. [Online]. Available: https://arxiv.org/abs/1805.03283

[13] Y. Cai, Y. Luo, S. Ghose, and O. Mutlu, "Read disturb errors in mlc nand flash memory: Characterization, mitigation, and recovery," in 2015 45th Annual IEEE/IFIP International Conference on Dependable Systems and Networks, 2015, pp. 438–449.

[14] Y. Cai, G. Yalcin, O. Mutlu, E. F. Haratsch, A. Cristal, O. S. Unsal, and K. Mai, "Flash correct-and-refresh: Retention-aware error management for increased flash memory lifetime," in 2012 IEEE 30th International Conference on Computer Design (ICCD), 2012, pp. 94–101.

[15] K. Chandrasekar, C. Weis, Y. Li, S. Goossens, M. Jung, O. Naji, B. Akesson, N. Wehn, and K. Goossens, "Drampower: Open-source dram power & energy estimation tool." [Online]. Available: http://www.drampower.info

[16] H.-J. Chi, C.-K. Lee, J. Park, J.-S. Heo, J. Jung, D. Lee, D.-H. Kim, D. Park, K. Kim, S.-Y. Kim, J. Park, H. Cho, S. Lim, Y. Choi, Y. Lim, D. Moon, G. Park, J.-H. Jang, K. Lee, I. Hwang, C. Kim, Y. Son, G.-Y. Kang, K. Park, S. Lee, S.-Y. Doo, C.-H. Shin, B. Na, J. Kwon, K. R. Kim, H. Choi, S.-K. Choi, S. Chang, W. Bae, H.-J. Kwon, Y.-S. Sohn, S.-J. Bae, K.-I. Park, and J.-B. Lee, "22.2 an 8.5gb/s/pin 12gb-lpddr5 sdram with a hybrid-bank architecture using skew-tolerant, low-power and speed-boosting techniques in a 2nd generation 10nm dram process," in 2020 IEEE International Solid-State Circuits Conference - (ISSCC), 2020, pp. 382–384.

[17] W. Cho, J. Jung, J. Kim, J. Ham, S. Lee, Y. Noh, D. Kim, W. Lee, K. Cho, K. Kim, H. Lee, S. Chai, E. Jo, H. Cho, J.-S. Kim, C. Kwon, C. Park, H. Nam, H. Won, T. Kim, K. Park, S. Oh, J. Ban, J. Park, J. Shin, T. Shin, J. Jang, J. Mun, J. Choi, H. Choi, S.-W. Choi, W. Park, D. Yoon, M. Kim, J. Lim, C. An, H. Shirr, H. Oh, H. Park, S. Shim, H. Huh, H. Choi, S. Lee, J. Sim, K. Gwon, J. Kim, W. Jeong, J. Choi, and K.-W. Jin, "A 1-tb, 4b/cell, 176-stacked-wl 3d-nand flash memory with improved read latency and a 14.8gb/mm2 density," in 2022 IEEE International Solid-State Circuits Conference (ISSCC), vol. 65, 2022, pp. 134–135.

[18] J. Choquette, "Nvidia hopper h100 gpu: Scaling performance," IEEE Micro, vol. 43, no. 3, pp. 9–17, 2023.

[19] M. Chun, J. Lee, M. Kim, J. Park, and J. Kim, "Rif: Improving read performance of modern ssds using an on-die early-retry engine," in 2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA), 2024, pp. 643–656.

[20] M. Chun, J. Lee, S. Lee, M. Kim, and J. Kim, "Pif: in-flash acceleration for data-intensive applications," in Proceedings of the 14th ACM Workshop on Hot Topics in Storage and File Systems, ser. HotStorage '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 106–112. [Online]. Available: https://doi.org/10.1145/3538643.3539742

[21] X. Dong, C. Xu, Y. Xie, and N. P. Jouppi, "Nvsim: A circuit-level performance, energy, and area model for emerging nonvolatile memory," IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, vol. 31, no. 7, pp. 994–1007, 2012.

[22] A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Yang, A. Fan, A. Goyal, A. Hartshorn, A. Yang, A. Mitra, A. Sravankumar, A. Korenev, A. Hinsvark, A. Rao, A. Zhang, A. Rodriguez, A. Gregerson, A. Spataru, B. Rozière, B. Biron, B. Tang, B. Chern, C. Caucheteux, C. Nayak, C. Bi, C. Marra, C. McConnell, C. Keller, C. Touret, C. Wu, C. Wong, C. C. Ferrer, C. Nikolaidis, D. Allonsius, D. Song, D. Pintz, D. Livshits, D. Esiobu, D. Choudhary, D. Mahajan, D. Garcia-Olano, D. Perino, D. Hupkes, E. Lakomkin, E. A. AlBadawy, E. Lobanova, E. Dinan, E. M. Smith, F. Radenovic, F. Zhang, G. Synnaeve, G. Lee, G. L. Anderson, G. Nail, G. Mialon, G. Pang, G. Cucurell, H. Nguyen, H. Korevaar, H. Xu, H. Touvron, I. Zarov, I. A. Ibarra, I. M. Kloumann, I. Misra, I. Evtimov, J. Copet, J. Lee, J. L. Geffert, J. Vranes, J. Park, J. Mahadeokar, J. Shah, J. van der Linde, J. Billock, J. Hong, J. Lee, J. Fu, J. Chi, J. Huang, J. Liu, J. Wang, J. Yu, J. Bitton, J. Spisak, J. Park, J. Rocca, J. Johnstun, J. Saxe, J.-Q. Jia, K. V. Alwala, K. Upasani, K. Plawiak, K. Li, K.-. neth Heafield, K. Stone, K. El-Arini, K. Iyer, K. Malik, K. Chiu, K. Bhalla, L. Rantala-Yeary, L. van der Maaten, L. Chen, L. Tan, L. Jenkins, L. Martin, L. Madaan, L. Malo, L. Blecher, L. Landzaat, L. de Oliveira, M. C. Muzzi, M. B. Pasupuleti, M. Singh, M. Paluri, M. Kardas, M. Oldham, M. Rita, M. Pavlova, M. H. M. Kambadur, M. Lewis, M. Si, M. K. Singh, M. Hassan, N. Goyal, N. Torabi, N. Bashlykov, N. Bogoychev, N. S. Chatterji, O. Duchenne, O. cCelebi, P. Alrassy, P. Zhang, P. Li, P. Vasić, P. Weng, P. Bhargava, P. Dubal, P. Krishnan, P. S. Koura, P. Xu, Q. He, Q. Dong, R. Srinivasan, R. Ganapathy, R. Calderer, R. S. Cabral, R. Stojnic, R. Raileanu, R. Girdhar, R. Patel, R. Sauvestre, R. Polidoro, R. Sumbaly, R. Taylor, R. Silva, R. Hou, R. Wang, S. Hosseini, S. Chennabasappa, S. Singh, S. Bell, S. S. Kim, S. Edunov, S. Nie, S. Narang, S. C. Raparthy, S. Shen, S. Wan, S. Bhosale, S. Zhang, S. Vandenhende, S. Batra, S. Whitman, S. Sootla, S. Collot, S. Gururangan, S. Borodinsky, T. Herman, T. Fowler, T. Sheasha, T. Georgiou, T. Scialom, T. Speckbacher, T. Mihaylov, T. Xiao, U. Karn, V. Goswami, V. Gupta, V. Ramanathan, V. Kerkez, V. Gonguet,

V. Do, V. Vogeti, V. Petrovic, W. Chu, W. Xiong, W. Fu, W. Meers, X. Martinet, X. Wang, X. E. Tan, X. Xie, X. Jia, X. Wang, Y. Goldschlag, Y. Gaur, Y. Babaei, Y. Wen, Y. Song, Y. Zhang, Y. Li, Y. Mao, Z. D. Coudert, Z. Yan, Z. Chen, Z. Papakipos, A. K. Singh, A. Grattafiori, A. Jain, A. Kelsey, A. Shajnfeld, A. Gangidi, A. Victoria, A. Goldstand, A. Menon, A. Sharma, A. Boesenberg, A. Vaughan, A. Baevski, A. Feinstein, A. Kallet, A. Sangani, A. Yunus, A. Lupu, A. Alvarado, A. Caples, A. Gu, A. Ho, A. Poulton, A. Ryan, A. Ramchandani, A. Franco, A. Saraf, A. Chowdhury, A. Gabriel, A. Bharambe, A. Eisenman, A. Yazdan, B. James, B. Maurer, B. Leonhardi, B. Huang, B. Loyd, B. D. Paola, B. Paranjape, B. Liu, B. Wu, B. Ni, B. Hancock, B. Wasti, B. Spence, B. Stojkovic, B. Gamido, B. Montalvo, C. Parker, C. Burton, C. Mejia, C. Wang, C. Kim, C. Zhou, C. Hu, C.-H. Chu, C. Cai, C. Tindal, C. Feichtenhofer, D. Civin, D. Beaty, D. Kreymer, S.-W. Li, D. Wyatt, D. Adkins, D. Xu, D. Testuggine, D. David, D. Parikh, D. Liskovich, D. Foss, D. Wang, D. Le, D. Holland, E. Dowling, E. Jamil, E. Montgomery, E. Presani, E. Hahn, E. Wood, E. Brinkman, E. Arcaute, E. Dunbar, E. Smothers, F. Sun, F. Kreuk, F. Tian, F. Ozgenel, F. Caggioni, F. Guzm'an, F. J. Kanayet, F. Seide, G. M. Florez, G. Schwarz, G. Badeer, G. Swee, G. Halpern, G. Thattai, G. Herman, G. G. Sizov, G. Zhang, G. Lakshminarayanan, H. Shojanazeri, H. Zou, H. Wang, H. Zha, H. Habeeb, H. Rudolph, H. Suk, H. Aspegren, H. Goldman, I. Molybog, I. Tufanov, I.-E. Veliche, I. Gat, J. Weissman, J. Geboski, J. Kohli, J. Asher, J.-B. Gaya, J. Marcus, J. Tang, J. Chan, J. Zhen, J. Reizenstein, J. Teboul, J. Zhong, J. Jin, J. Yang, J. Cummings, J. Carvill, J. Shepard, J. McPhie, J. Torres, J. Ginsburg, J. Wang, K. Wu, U. KamHou, K. Saxena, K. Prasad, K. Khandelwal, K. Zand, K. Matosich, K. Veeraraghavan, K. Michelena, K. Li, K. Huang, K. Chawla, K. Lakhotia, K. Huang, L. Chen, L. Garg, A. Lavender, L. Silva, L. Bell, L. Zhang, L. Guo, L. Yu, L. Moshkovich, L. Wehrstedt, M. Khabsa, M. Avalani, M. Bhatt, M. Tsimpoukelli, M. Mankus, M. Hasson, M. Lennie, M. Reso, M. Groshev, M. Naumov, M. Lathi, M. Keneally, M. L. Seltzer, M. Valko, M. Restrepo, M. Patel, M. Vyatskov, M. Samvelyan, M. Clark, M. Macey, M. Wang, M. J. Hermoso, M. Metanat, M. Rastegari, M. Bansal, N. Santhanam, N. Parks, N. White, N. Bawa, N. Singhal, N. Egebo, N. Usunier, N. P. Laptev, N. Dong, N. Zhang, N. Cheng, O. Chernoguz, O. Hart, O. Salpekar, O. Kalinli, P. Kent, P. Parekh, P. Saab, P. Balaji, P. Rittner, P. Bontrager, P. Roux, P. Dollár, P. Zvyagina, P. Ratanchandani, P. Yuvraj, Q. Liang, R. Alao, R. Rodriguez, R. Ayub, R. Murthy, R. Nayani, R. Mitra, R. Li, R. Hogan, R. Battey, R. Wang, R. Maheswari, R. Howes, R. Rinott, S. J. Bondu, S. Datta, S. Chugh, S. Hunt, S. Dhillon, S. Sidorov, S. Pan, S. Verma, S. Yamamoto, S. Ramaswamy, S. Lindsay, S. Feng, S. Lin, S. C. Zha, S. Shankar, S. Zhang, S. Wang, S. Agarwal, S. Sajuyigbe, S. Chintala, S. Max, S. Chen, S. Kehoe, S. Satterfield, S. Govindaprasad, S. Gupta, S.-B. Cho, S. Virk, S. Subramanian, S. Choudhury, S. Goldman, T. Remez, T. Glaser, T. Best, T. Kohler, T. Robinson, T. Li, T. Zhang, T. Matthews, T. Chou, T. Shaked, V. Vontimitta, V. Ajayi, V. Montanez, V. Mohan, V. S. Kumar, V. Mangla, V. Ionescu, V. A. Poenaru, V. T. Mihailescu, V. Ivanov, W. Li, W. Wang, W. Jiang, W. Bouaziz, W. Constable, X. Tang, X. Wang, X. Wu, X. Wang, X. Xia, X. Wu, X. Gao, Y. Chen, Y. Hu, Y. Jia, Y. Qi, Y. Li, Y. Zhang, Y. Zhang, Y. Adi, Y. Nam, Y. Wang, Y. Hao, Y. Qian, Y. He, Z. Rait, Z. DeVito, Z. Rosnbrick, Z. Wen, Z. Yang, and Z. Zhao, "The llama 3 herd of models," ArXiv, vol. abs/2407.21783, 2024. [Online]. Available: https://arxiv.org/abs/2407.21783

[23] A. Elmeleegy, S. Raj, B. Slechta, and V. Mehta, "Nvidia technical blog: Demystifying ai inference deployments for trillion parameter large language models." [Online]. Available: https://developer.nvidia.com/blog/demystifying-ai-inference-deployments-for-trillion-parameter-large-language-models/

[24] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "OPTQ: Accurate quantization for generative pre-trained transformers," in The Eleventh International Conference on Learning Representations, 2023. [Online]. Available: https://openreview.net/forum?id=tcbBPnfwxS

[25] D. Gouk, M. Kwon, J. Zhang, S. Koh, W. Choi, N. S. Kim, M. Kandemir, and M. Jung, "Amber: Enabling precise full-system simulation with detailed modeling of all ssd resources," in 2018 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO), 2018, pp. 469–481.

[26] M. Greenberg, "Modifying a dram controller to support lpddr2-nvm," in Flash Memory Summit (2009), 2009. [Online]. Available: https://files.futurememorystorage.com/proceedings/2009/20090812_S106_Greenberg.pdf

[27] Y. Gu, L. Dong, F. Wei, and M. Huang, "Minillm: Knowledge distillation of large language models," in ICLR, 2024. [Online]. Available: https://openreview.net/forum?id=5h0qf7IBZZ

[28] T. Gunter, Z. Wang, C. Wang, R. Pang, A. Narayanan, A. Zhang, B. Zhang, C. Chen, C.-C. Chiu, D. Qiu, D. Gopinath, D. A. Yap, D. Yin, F. Nan, F. Weers, G. Yin, H. Huang, J. Wang, J. Lu, J. Peebles, K. Ye, M. Lee, N. Du, Q. Chen, Q. Keunebroek, S. Wiseman, S. Evans, T. Lei, V. Rathod, X. Kong, X. Du, Y. Li, Y. Wang, Y. Gao, Z. Ahmed, Z. Xu, Z. Lu, A. Rashid, A. M. Jose, A. Doane, A. Bencomo, A. Vanderby, A. Hansen, A. Jain, A. M. Anupama, A. Kamal, B. Wu, C. Brum, C. Maalouf, C. Erdenebileg, C. Dulhanty, D. Moritz, D. Kang, E. Jimenez, E. Ladd, F. Shi, F. Bai, F. Chu, F. Hohman, H. Kotek, H. G. Coleman, J. Li, J. Bigham, J. Cao, J. Lai, J. Cheung, J. Shan, J. Zhou, J. Li, J. Qin, K. Singh, K. Vega, K. Zou, L. Heckman, L. Gardiner, M. Bowler, M. Cordell, M. Cao, N. Hay, N. Shahdadpuri, O. Godwin, P. Dighe, P. Rachapudi, R. Tantawi, R. Frigg, S. Davarnia, S. Shah, S. Guha, S. Sirovica, S. Ma, S. Ma, S. Wang, S. Kim, S. Jayaram, V. Shankar, V. Paidi, V. Kumar, X. Wang, X. Zheng, W. Cheng, Y. Shrager, Y. Ye, Y. Tanaka, Y. Guo, Y. Meng, Z. T. Luo, Z. Ouyang, A. Aygar, A. Wan, A. Walkingshaw, A. Narayanan, A. Lin, A. Farooq, B. Ramerth, C. Reed, C. Bartels, C. Chaney, D. Riazati, E. L. Yang, E. Feldman, G. Hochstrasser, G. Seguin, I. Belousova, J. Pelemans, K. Yang, K. A. Vahid, L. Cao, M. Najibi, M. Zuliani, M. Horton, M. Cho, N. Bhendawade, P. Dong, P. Maj, P. Agrawal, Q. Shan, Q. Fu, R. Poston, S. Xu, S. Liu, S. Rao, T. Heeramun, T. Merth, U. Rayala, V. Cui, V. R. Sridhar, W. Zhang, W. Zhang, W. Wu, X. Zhou, X. Liu, Y. Zhao, Y. Xia, Z. Ren, and Z. Ren, "Apple intelligence foundation language models," 2024. [Online]. Available: https://arxiv.org/abs/2407.21075

[29] H. Ham, W. Yang, Y. Shin, O. Woo, G. Heo, S. Lee, J. Park, and G. Kim, "Onnxim: A fast, cycle-level multi-core npu simulator," arXiv preprint arXiv:2406.08051, 2024.

[30] M. He, C. Song, I. Kim, C. Jeong, S. Kim, I. Park, M. Thottethodi, and T. N. Vijaykumar, "Newton: A dram-maker's accelerator-in-memory (aim) architecture for machine learning," in 2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO), 2020, pp. 372–385.

[31] G. Heo, S. Lee, J. Cho, H. Choi, S. Lee, H. Ham, G. Kim, D. Mahajan, and J. Park, "Neupims: Npu-pim heterogeneous acceleration for batched llm inferencing," in Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3, ser. ASPLOS '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 722–737. [Online]. Available: https://doi.org/10.1145/3620666.3651380

[32] S. H. J. Huang and Y. D. Chen, "Advanced patterning techniques for 3d nand devices." [Online]. Available: https://semiengineering.com/advanced-patterning-techniques-for-3d-nand-devices/

[33] Infineon, "Infineon enables next-generation automotive e/e architectures with industry's first lpddr flash memory," 2023. [Online]. Available: https://www.infineon.com/cms/en/about-infineon/press/market-news/2023/INFATV202304-092.html

[34] Infineon, "Semper™ x1 lpddr flash," 2023. [Online]. Available: https://www.infineon.com/cms/en/product/memories/nor-flash/semper-x1-lpddr-flash/

[35] Intel, "Intel's lunar lake processors arriving q3 2024." [Online]. Available: https://www.intel.com/content/www/us/en/newsroom/news/intels-lunar-lake-processors-arriving-q3-2024.html

[36] A. Jain, H. Vaghasiya, and J. N. Tripathi, "Efficient selection and placement of in-package decoupling capacitors using matrix-based evolutionary computation," IEEE Open Journal of Nanotechnology, vol. 2, pp. 191–200, 2021.

[37] H. Jang, J. Song, J. Jung, J. Park, Y. Kim, and J. Lee, "Smart-infinity: Fast large language model training using near-storage processing on a real system," in 2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA), 2024, pp. 345–360.

[38] I. Jani, D. Lattard, P. Vivet, L. Arnaud, S. Cheramy, E. Beigné, A. Farcy, J. Jourdon, Y. Henrion, E. Deloffre, and H. Bilgen, "Characterization of fine pitch hybrid bonding pads using electrical misalignment test vehicle," in 2019 IEEE 69th Electronic Components and Technology Conference (ECTC), 2019, pp. 1926–1932.

[39] JCET, "Fine pitch ball grid array - stacked die," accessed on July 24th, 2024. [Online]. Available: https://www.jcetglobal.com/uploads/FBGA-SD%20-%20Fine-Pitch%20Ball%20Grid%20Array%20with%20Stacked%20Die.pdf

[40] JCET, "Plastic ball grid array - stacked die," accessed on July 24th, 2024. [Online]. Available: https://www.jcetglobal.com/uploads/PBGA-SD_22Dec2021.pdf

[41] JEDEC, "Fine pitch thick square ball grid array package (bf-bga)," accessed on July 24th, 2024. [Online]. Available: https://www.jedec.org/sites/default/files/docs/DRegistration4-27A.pdf
[42] JEDEC, "Jedec announces publication of lpddr2 standard for low power memory devices," 2009. [Online]. Available: https://www.jedec.org/news/pressreleases/jedec-announces-publication-lpddr2-standard-low-power-memory-devices
[43] JEDEC, "Low power double data rate 2 (lpddr2)," 2009. [Online]. Available: https://www.jedec.org/standards-documents/docs/jesd209-2f
[44] Y. Jin, S. Kim, T. J. Ham, and J. W. Lee, "Architecting a flash-based storage system for low-cost inference of extreme-scale dnns," IEEE Transactions on Computers, vol. 71, no. 12, pp. 3153–3164, 2022.
[45] M. Jung, W. Choi, S. Gao, E. H. Wilson III, D. Donofrio, J. Shalf, and M. T. Kandemir, "Nandflashsim: High-fidelity, microarchitecture-aware nand flash memory simulation," ACM Trans. Storage, vol. 12, no. 2, jan 2016. [Online]. Available: https://doi.org/10.1145/2700310
[46] M. Jung, J. Zhang, A. Abulila, M. Kwon, N. Shahidi, J. Shalf, N. S. Kim, and M. Kandemir, "Simplessd: Modeling solid state drives for holistic system simulation," IEEE Computer Architecture Letters, vol. 17, no. 1, pp. 37–41, 2018.
[47] J. Kim, M. Kang, Y. Han, Y.-G. Kim, and L.-S. Kim, "Optimstore: In-storage optimization of large scale dnns with on-die processing," in 2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA), 2023, pp. 611–623.
[48] S. Kim, Y. Jin, G. Sohn, J. Bae, T. J. Ham, and J. W. Lee, "Behemoth: A flash-centric training accelerator for extreme-scale DNNs," in 19th USENIX Conference on File and Storage Technologies (FAST 21). USENIX Association, Feb. 2021, pp. 371–385. [Online]. Available: https://www.usenix.org/conference/fast21/presentation/kim
[49] KIOXIA, "New storage class memory solution accelerates non-relational database performance," 2022, accessed on July 1st, 2024. [Online]. Available: https://americas.kioxia.com/content/dam/kioxia/en-us/business/ssd/enterprise-ssd/asset/KIOXIA_FL6_Application_Brief.pdf
[50] T. Kouchi, N. Kumazaki, M. Yamaoka, S. Bushnaq, T. Kodama, Y. Ishizaki, Y. Deguchi, A. Sugahara, A. Imamoto, N. Asaoka, R. Isomura, T. Handa, J. Sato, H. Komai, A. Okuyama, N. Kanagawa, Y. Kajiyama, Y. Terada, H. Ohnishi, H. Yabe, C. Hsu, M. Kakoi, and M. Yoshihara, "13.5 a 128gb 1b/cell 96-word-line-layer 3d flash memory to improve random read latency with tprog=75µs and tr=4µs," in 2020 IEEE International Solid-State Circuits Conference - (ISSCC), 2020, pp. 226–228.
[51] J. Kwak, S. Lee, K. Park, J. Jeong, and Y. H. Song, "Cosmos+ openssd: Rapid prototype for flash storage systems," ACM Trans. Storage, vol. 16, no. 3, Jul. 2020. [Online]. Available: https://doi.org/10.1145/3385073
[52] Y. Kwon, K. Vladimir, N. Kim, W. Shin, J. Won, M. Lee, H. Joo, H. Choi, G. Kim, B. An, J. Kim, J. Lee, I. Kim, J. Park, C. Park, Y. Song, B. Yang, H. Lee, S. Kim, D. Kwon, S. Lee, K. Kim, S. Oh, J. Park, G. Hong, D. Ka, K. Hwang, J. Park, K. Kang, J. Kim, J. Jeon, M. Lee, M. Shin, M. Shin, J. Cha, C. Jung, K. Chang, C. Jeong, E. Lim, I. Park, J. Chun, and S. Hynix, "System architecture and software stack for gddr6-aim," in 2022 IEEE Hot Chips 34 Symposium (HCS), 2022, pp. 1–25.
[53] C. Lee, W. Shin, D. J. Kim, Y. Yu, S.-J. Kim, T. Ko, D. Seo, J. Park, K. Lee, S. Choi, N. Kim, V. G, A. George, V. V, D. Lee, K. Choi, C. Song, D. Kim, I. Choi, I. Jung, Y. H. Song, and J. Han, "Nvdimm-c: A byte-addressable non-volatile memory module for compatibility with standard ddr memory interfaces," in 2020 IEEE International Symposium on High Performance Computer Architecture (HPCA), 2020, pp. 502–514.
[54] H. Lee, M. Kim, D. Min, J. Kim, J. Back, H. Yoo, J.-H. Lee, and J. Kim, "3d-fpim: An extreme energy-efficient dnn acceleration system using 3d nand flash-based in-situ pim unit," in 2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO), 2022, pp. 1359–1376.
[55] J. Lee, H. Kim, K. Kyung, M. You, H. Lee, K. Park, and B. Chung, "Design, analysis, and optimization of ddr2 memory power delivery network," in 2007 IEEE Electrical Performance of Electronic Packaging, 2007, pp. 87–90.
[56] S. Lee, K. Kim, S. Oh, J. Park, G. Hong, D. Ka, K. Hwang, J. Park, K. Kang, J. Kim, J. Jeon, N. Kim, Y. Kwon, K. Vladimir, W. Shin, J. Won, M. Lee, H. Joo, H. Choi, J. Lee, D. Ko, Y. Jun, K. Cho, I. Kim, C. Song, C. Jeong, D. Kwon, J. Jang, I. Park, J. Chun, and J. Cho, "A 1ynm 1.25v 8gb, 16gb/s/pin gddr6-based accelerator-in-memory supporting 1tflops mac operation and various activation functions for deep-learning applications," in 2022 IEEE International Solid-State Circuits Conference (ISSCC), vol. 65, 2022, pp. 1–3.
[57] Y. Lee, H. Kim, and M. Rhu, "Presto: An in-storage data preprocessing system for training recommendation models," in 2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA), 2024, pp. 340–353.
[58] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," in Proceedings of the 40th International Conference on Machine Learning, ser. ICML'23. JMLR.org, 2023.
[59] L. Li, S. Qian, J. Lu, L. Yuan, R. Wang, and Q. Xie, "Transformer-lite: High-efficiency deployment of large language models on mobile phone gpus," 2024. [Online]. Available: https://arxiv.org/abs/2403.20041
[60] H. Luo, Y. C. Tuğrul, F. N. Bostancı, A. Olgun, A. G. Yağlıkçı, and O. Mutlu, "Ramulator 2.0: A modern, modular, and extensible dram simulator," IEEE Computer Architecture Letters, vol. 23, no. 1, pp. 112–116, 2024.
[61] X. Ma, G. Fang, and X. Wang, "Llm-pruner: On the structural pruning of large language models," in Advances in Neural Information Processing Systems, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, Eds., vol. 36. Curran Associates, Inc., 2023, pp. 21 702–21 720. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/file/44956951349095f74492a5471128a7e0-Paper-Conference.pdf
[62] G. Malavena, A. L. Lacaita, A. S. Spinelli, and C. Monzio Compagnoni, "Investigation and compact modeling of the time dynamics of the gidl-assisted increase of the string potential in 3-d nand flash arrays," IEEE Transactions on Electron Devices, vol. 65, no. 7, pp. 2804–2811, 2018.
[63] X. Miao, G. Oliaro, Z. Zhang, X. Cheng, Z. Wang, Z. Zhang, R. Y. Y. Wong, A. Zhu, L. Yang, X. Shi, C. Shi, Z. Chen, D. Arfeen, R. Abhyankar, and Z. Jia, "Specinfer: Accelerating large language model serving with tree-based speculative inference and verification," in Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3, ser. ASPLOS '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 932–949. [Online]. Available: https://doi.org/10.1145/3620666.3651335
[64] R. Micheloni, S. Aritome, and L. Crippa, "Array architectures for 3-d nand flash memories," Proceedings of the IEEE, vol. 105, no. 9, pp. 1634–1649, 2017.
[65] R. Micheloni, L. Crippa, and A. Marelli, Inside NAND flash memories, 01 2010.
[66] R. Micheloni, A. Marelli, and K. Eshghi, Inside Solid State Drives (SSDs), 2nd ed. Springer Publishing Company, Incorporated, 2018.
[67] Micron, "Lpddr5/lpddr5x datasheet."
[68] J. Morra, "Infineon rolls out first nor flash memory with lpddr inside," 2023. [Online]. Available: https://www.electronicdesign.com/markets/automotive/article/21264779/electronic-design-infineon-rolls-out-first-nor-flash-memory-with-lpddr-inside
[69] V. Moschiano, "Tutorial: 3d flash memory from technology to the system: past, present and future developments," in 2024 IEEE International Solid-State Circuits Conference (ISSCC), 2024.
[70] NVIDIA, "Nvidia hopper architecture in-depth," 2022. [Online]. Available: https://developer.nvidia.com/blog/nvidia-hopper-architecture-in-depth/
[71] OpenAI, "Chatgpt pricing," accessed on October 13th, 2024. [Online]. Available: https://openai.com/chatgpt/pricing/
[72] Y. Ouyang, S. Yang, D. Yin, X. Huang, Z. Wang, S. Yang, K. Han, and Z. Xia, "Excellent reliability of xtacking™ bonding interface," in 2021 IEEE International Reliability Physics Symposium (IRPS), 2021, pp. 1–6.
[73] N. Papandreou, N. Ioannou, T. Parnell, R. Pletka, M. Stanisavljevic, R. Stoica, S. Tomic, and H. Pozidis, "Reliability of 3d nand flash memory with a focus on read voltage calibration from a system aspect," in 2019 19th Non-Volatile Memory Technology Symposium (NVMTS), 2019, pp. 1–4.
[74] N. Papandreou, H. Pozidis, N. Ioannou, T. Parnell, R. Pletka, M. Stanisavljevic, R. Stoica, S. Tomic, P. Breen, G. Tressler, A. Fry, T. Fisher, and A. Walls, "Open block characterization and read voltage calibration of 3d qlc nand flash," in 2020 IEEE International Reliability Physics Symposium (IRPS), 2020, pp. 1–6.
[75] J. Park, J. Choi, K. Kyung, M. J. Kim, Y. Kwon, N. S. Kim, and J. H. Ahn, "Attacc! unleashing the power of pim for batched transformer-based generative model inference," in Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ser. ASPLOS '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 103–119. [Online]. Available: https://doi.org/10.1145/3620665.3640422
[76] J. Park, M. Kim, M. Chun, L. Orosa, J. Kim, and O. Mutlu, "Reducing solid-state drive read latency by optimizing read-retry," in Proceedings of the 26th ACM International Conference on Architectural Support

for Programming Languages and Operating Systems, ser. ASPLOS '21. New York, NY, USA: Association for Computing Machinery, 2021, p. 702–716. [Online]. Available: https://doi.org/10.1145/3445814.3446719

[77] S.-S. Park, K. Kim, J. So, J. Jung, J. Lee, K. Woo, N. Kim, Y. Lee, H. Kim, Y. Kwon, J. Kim, J. Lee, Y. Cho, Y. Tai, J. Cho, H. Song, J. H. Ahn, and N. S. Kim, "An lpddr-based cxl-pnm platform for tco-efficient inference of transformer-based large language models," in 2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA), 2024, pp. 970–982.

[78] D. Patel, "Apple m2 die shot and architecture analysis – big cost increase and a15 based ip." [Online]. Available: https://www.semianalysis.com/p/apple-m2-die-shot-and-architecture

[79] M. Poremba, S. Mittal, D. Li, J. S. Vetter, and Y. Xie, "Destiny: A tool for modeling emerging 3d nvm and edram caches," in 2015 Design, Automation & Test in Europe Conference & Exhibition (DATE), 2015, pp. 1543–1546.

[80] E. Qin, A. Samajdar, H. Kwon, V. Nadella, S. Srinivasan, D. Das, B. Kaul, and T. Krishna, "Sigma: A sparse and irregular gemm accelerator with flexible interconnects for dnn training," in 2020 IEEE International Symposium on High Performance Computer Architecture (HPCA), 2020, pp. 58–70.

[81] Y. Qiu, W. Yin, and L. Wang, "A high-performance open-channel open-way nand flash controller architecture," in 2021 31st International Conference on Field-Programmable Logic and Applications (FPL), 2021, pp. 91–98.

[82] Qualcomm, "Snapdragon x elite." [Online]. Available: https://www.qualcomm.com/products/mobile/snapdragon/pcs-and-tablets/snapdragon-x-elite#Overview

[83] M. Seo, X. T. Nguyen, S. J. Hwang, Y. Kwon, G. Kim, C. Park, I. Kim, J. Park, J. Kim, W. Shin, J. Won, H. Choi, K. Kim, D. Kwon, C. Jeong, S. Lee, Y. Choi, W. Byun, S. Baek, H.-J. Lee, and J. Kim, "Ianus: Integrated accelerator based on npu-pim unified memory system," in Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3, ser. ASPLOS '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 545–560. [Online]. Available: https://doi.org/10.1145/3620666.3651324

[84] J. Shin, S. Kim, K. Sung, H. Jung, J. Song, W. Choi, H. Ko, J. Choi, and S. Lee, "Reinforcement learning-based decap optimization method for high-performance solid-state drive," in 2021 IEEE International Joint EMC/SI/PI and EMC Europe Symposium, 2021, pp. 718–721.

[85] L. Smith, R. Anderson, D. Forehand, T. Pelc, and T. Roy, "Power distribution system design methodology and capacitor selection for modern cmos technology," IEEE Transactions on Advanced Packaging, vol. 22, no. 3, pp. 284–291, 1999.

[86] J. Song, C. Ryu, S. Park, D. Jung, J. Shin, and Y. Ku, "Optimal power distribution network design for high-performance solid-state-drive based on novel target-impedance extraction method," in 2021 IEEE International Joint EMC/SI/PI and EMC Europe Symposium, 2021, pp. 163–167.

[87] A. Stillmaker and B. Baas, "Scaling equations for the accurate prediction of cmos device performance from 180nm to 7nm," Integration, vol. 58, pp. 74–81, 2017. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0167926017300755

[88] W. Sun, Z. Li, S. Yin, S. Wei, and L. Liu, "Abc-dimm: Alleviating the bottleneck of communication in dimm-based near-memory processing with inter-dimm broadcast," in 2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA), 2021, pp. 237–250.

[89] A. Tavakkol, J. Gómez-Luna, M. Sadrosadati, S. Ghose, and O. Mutlu, "MQSim: A framework for enabling realistic studies of modern Multi-Queue SSD devices," in 16th USENIX Conference on File and Storage Technologies (FAST 18). Oakland, CA: USENIX Association, Feb. 2018, pp. 49–66. [Online]. Available: https://www.usenix.org/conference/fast18/presentation/tavakkol

[90] TechPowerUp, "Ssd specs database." [Online]. Available: https://www.techpowerup.com/ssd-specs/

[91] TechPowerUp, "Zhitai tipro7000 2 tb." [Online]. Available: https://www.techpowerup.com/ssd-specs/zhitai-tipro7000-2-tb.d1192

[92] the Open NAND Flash Interface Workgroup, "Open nand flash interface specification," 2017, accessed on July 1st, 2024. [Online]. Available: https://onfi.org/files/onfi_4_1_gold.pdf

[93] R. Thoppilan, D. D. Freitas, J. Hall, N. Shazeer, A. Kulshreshtha, H.-T. Cheng, A. Jin, T. Bos, L. Baker, Y. Du, Y. Li, H. Lee, H. S. Zheng, A. Ghafouri, M. Menegali, Y. Huang, M. Krikun, D. Lepikhin, J. Qin, D. Chen, Y. Xu, Z. Chen, A. Roberts, M. Bosma, V. Zhao, Y. Zhou, C.-C. Chang, I. Krivokon, W. Rusch, M. Pickett, P. Srinivasan, L. Man, K. Meier-Hellstern, M. R. Morris, T. Doshi, R. D. Santos, T. Duke, J. Soraker, B. Zevenbergen, V. Prabhakaran, M. Diaz, B. Hutchinson, K. Olson, A. Molina, E. Hoffman-John, J. Lee, L. Aroyo, R. Rajakumar, A. Butryna, M. Lamm, V. Kuzmina, J. Fenton, A. Cohen, R. Bernstein, V. M. Kurzweil, B. Aguera-Arcas, C. Cui, M. Croak, E. Chi, and Q. Le, "Lamda: Language models for dialog applications," 2022. [Online]. Available: https://arxiv.org/abs/2201.08239

[94] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample, "Llama: Open and efficient foundation language models," 2023. [Online]. Available: https://arxiv.org/abs/2302.13971

[95] A. Tseng, J. Chee, Q. Sun, V. Kuleshov, and C. D. Sa, "QuIP$\#$: Even better LLM quantization with hadamard incoherence and lattice codebooks," in Forty-first International Conference on Machine Learning, 2024. [Online]. Available: https://openreview.net/forum?id=9BrydUVcoe

[96] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in Proceedings of the 31st International Conference on Neural Information Processing Systems, ser. NIPS'17. Red Hook, NY, USA: Curran Associates Inc., 2017, p. 6000–6010.

[97] O. Villa, D. R. Johnson, M. Oconnor, E. Bolotin, D. Nellans, J. Luitjens, N. Sakharnykh, P. Wang, P. Micikevicius, A. Scudiero, S. W. Keckler, and W. J. Dally, "Scaling the power wall: A path to exascale," in SC '14: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, 2014, pp. 830–841.

[98] M. Xia, T. Gao, Z. Zeng, and D. Chen, "Sheared LLaMA: Accelerating language model pre-training via structured pruning," in The Twelfth International Conference on Learning Representations, 2024. [Online]. Available: https://openreview.net/forum?id=09iOdaeOzp

[99] X. Xie, Z. Liang, P. Gu, A. Basak, L. Deng, L. Liang, X. Hu, and Y. Xie, "Spacea: Sparse matrix vector multiplication on processing-in-memory accelerator," in 2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA), 2021, pp. 570–583.

[100] M. Yan, S. Agarwal, and S. Venkataraman, "Decoding speculative decoding," 2024. [Online]. Available: https://arxiv.org/abs/2402.01528

[101] D. B. L. Yolanda, "Wafer to wafer bonding to increase memory density," in 2022 China Semiconductor Technology International Conference (CSTIC), 2022, pp. 1–4.

[102] H. Yoo, Y. Lee, and I.-C. Park, "7.3 gb/s universal bch encoder and decoder for ssd controllers," in 2014 19th Asia and South Pacific Design Automation Conference (ASP-DAC), 2014, pp. 37–38.

[103] S. Yun, H. Nam, J. Park, B. Kim, J. H. Ahn, and E. Lee, "Grande: Efficient near-data processing architecture for graph neural networks," IEEE Transactions on Computers, pp. 1–14, 2023.

[104] J. Zhang, M. Kwon, D. Gouk, S. Koh, N. S. Kim, M. T. Kandemir, and M. Jung, "Revamping storage class memory with hardware automated memory-over-storage solution," in Proceedings of the 48th Annual International Symposium on Computer Architecture, ser. ISCA '21. IEEE Press, 2021, p. 762–775. [Online]. Available: https://doi.org/10.1109/ISCA52012.2021.00065

[105] R. Zhang, M. R. Stan, and K. Skadron, "Hotspot 6.0: Validation, acceleration and extension," 2015, tech. Report. [Online]. Available: https://www.cs.virginia.edu/~skadron/Papers/HotSpot60_TR.pdf

[106] S. Zhang, S. Roller, N. Goyal, M. Artetxe, M. Chen, S. Chen, C. Dewan, M. Diab, X. Li, X. V. Lin, T. Mihaylov, M. Ott, S. Shleifer, K. Shuster, D. Simig, P. S. Koura, A. Sridhar, T. Wang, and L. Zettlemoyer, "Opt: Open pre-trained transformer language models," 2022. [Online]. Available: https://arxiv.org/abs/2205.01068

[107] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, and H. Zhang, "Distserve: Disaggregating prefill and decoding for goodput-optimized large language model serving," in 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI), 2024.

[108] Z. Zhou, C. Li, F. Yang, and G. Sun, "Dimm-link: Enabling efficient inter-dimm communication for near-memory processing," in 2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA), 2023, pp. 302–316.

[109] C. Zitlaw, "Nvm on the lpddr interface," 2024. [Online]. Available: https://www.jedec.org/sites/default/files/Final_Cliff_Zitlaw_Infineon_NVM_on_the_LPDDR_Interface_2024final_2.pdf

[110] J. Zou, Q. Xu, J. Luo, R. Wang, R. Huang, and Y. Wang, "Predictive 3-d modeling of parasitic gate capacitance in gate-all-around cylindrical silicon nanowire mosfets," IEEE Transactions on Electron Devices, vol. 58, no. 10, pp. 3379–3387, 2011.